



SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES CHENNAI-
602105

**Design and Implementation of an RF Envelope
Detector and RF Signal Monitoring System Using a Schottky Diode**

A LAB REPORT
Submitted in the partial fulfilment for the award of the degree of
BACHELOR OF ENGINEERING
IN
ELECTRONICS AND COMMUNICATIONS ENGINEERING

FOR THE COURSE
ECA5610 : 5G / 6G Communication Systems

Submitted by
L. Vivekananda Reddy (192412380)

Under the Guidance of
Dr. M M Ramakrishna

AUGUST 2026

ECA 56 -LIST OF EXPERIMENTS

Exp. No.	Experiment	Date	PG no	Marks	Signature
1	Simulation of FDMA in MATLAB	14/7/2026	4 - 7		
2	Implementing TDMA Slot Allocation in GSM	16/7/2026	8 - 11		
3	Spectrum Analysis of Sub-1 GHz, Mid-band & mmWave by plotting propagation loss	21/7/2026	12 - 15		
4	OFDMA Resource Block Allocation	23/7/2026	16 - 20		
5	MIMO System Simulation with BER Analysis	25/7/2026	21 - 25		
6	5G NR Frame Structure Analysis with Different Subcarrier Spacing	28/7/2026	26 - 29		
7	Analog Beamforming	30/7/2026	30 - 33		
8	Steering Angle Variation in MIMO Beamforming	1/8/2026	34 - 37		
9	Massive MIMO Capacity vs SNR and Gain	4/8/2026	38 - 40		
10	5G Network Slicing Environment	6/8/2026	42 - 45		
11	Implementation of IoT-based Smart Agriculture using MATLAB	8/8/2026	46 - 49		
12	NB-IoT vs LTE-M Throughput Simulation	11/8/2026	50 - 53		
13	V2X Latency Comparison	13/8/2026	54 - 57		
14	5G Throughput vs SNR with Modulation Labels	18/8/2026	58 - 62		
15	AI-based Traffic Prediction for 5G Networks	20/8/2026	63 - 66		

ECA 5610 -LIST OF EXPERIMENTS

Exp. No.	Experiment	Duration (Expt. Conduct + Evaluation)	CO	BL	WK	Type of Expt.
1	Simulation of FDMA in MATLAB	45+ 15 min	CO1	L3	K3	Simulation
2	Implementing TDMA Slot Allocation in GSM	45+ 15 min	CO1	L3	K3	Simulation
3	Spectrum Analysis of Sub-1 GHz, Mid-band & mmWave by plotting propagation loss	45+ 15 min	CO1	L4	K4	Simulation & Analysis
4	OFDMA Resource Block Allocation	45+ 15 min	CO2	L3	K3	Simulation
5	MIMO System Simulation with BER Analysis	45+ 15 min	CO2	L4	K4	Simulation & Analysis
6	5G NR Frame Structure Analysis with Different Subcarrier Spacing	45+ 15 min	CO2	L4	K4	Simulation
7	Analog Beamforming	45+ 15 min	CO3	L4	K4	Simulation
8	Steering Angle Variation in MIMO Beamforming	45+ 15 min	CO3	L4	K4	Simulation & Analysis
9	Massive MIMO Capacity vs SNR and Gain	45+ 15 min	CO3	L5	K5	Simulation & Performance Analysis
10	5G Network Slicing Environment	45+ 15 min	CO4	L4	K4	Design & Simulation
11	Implementation of IoT-based Smart Agriculture using MATLAB	45+ 15 min	CO4	L6	K6	Design & Implementation
12	NB-IoT vs LTE-M Throughput Simulation	45+ 15 min	CO4	L5	K5	Comparative Simulation
13	V2X Latency Comparison	45+ 15 min	CO5	L5	K5	Simulation & Analysis
14	5G Throughput vs SNR with Modulation Labels	45+ 15 min	CO5	L4	K4	Performance Simulation
15	AI-based Traffic Prediction for 5G Networks	45+ 15 min	CO5	L6	K6	AI-Based Simulation

Ex.No:1: Simulation of FDMA in MATLAB

Aim: To simulate a Frequency Division Multiple Access (FDMA) system using MATLAB and analyze its communication performance in terms of channel bandwidth, data rate, and signal-to-noise ratio (SNR).

Objective 1:

To simulate an FDMA system by allocating **individual frequency channels** to multiple users and determine the **channel bandwidth** assigned to each user.

Output Parameter: Channel Bandwidth (kHz)

Objective 2:

To calculate the **data rate** achieved by each user in the FDMA system based on the allocated channel bandwidth.

Output Parameter: Data Rate (kbps)

Objective 3:

To evaluate the **communication quality** of the FDMA system by calculating the **Signal-to-Noise Ratio (SNR)** for different users.

Output Parameter: Signal-to-Noise Ratio (SNR) (dB)

Procedure

1. Open **MATLAB** and create a new script file.
2. Define the total available bandwidth, number of FDMA users, carrier frequencies, signal power, and noise power.
3. Allocate a unique frequency channel to each user and calculate the **channel bandwidth**.
4. Compute the **data rate** for each user based on the allocated bandwidth and modulation scheme.
5. Calculate the **Signal-to-Noise Ratio (SNR)** using the signal and noise power values.
6. Plot the transmitted signals of all FDMA users using different carrier frequencies.
7. Display the calculated **channel bandwidth, data rate, and SNR** in the MATLAB Command Window.
8. Analyze the results to verify independent channel allocation and evaluate the overall performance of

the FDMA system. **Program (Matlab Code)** clc; clear; close all; fs=1000; t=0:1/fs:1; TBW=600;

Users=3;

CBW=TBW/Users; fc=[100

300 500];

m=sin(2*pi*10*t);

s1=m.*cos(2*pi*fc(1)*t);

s2=m.*cos(2*pi*fc(2)*t);

s3=m.*cos(2*pi*fc(3)*t); subplot(2,1,1)

plot(t,s1,'r',t,s2,'g',t,s3,'b'),grid on

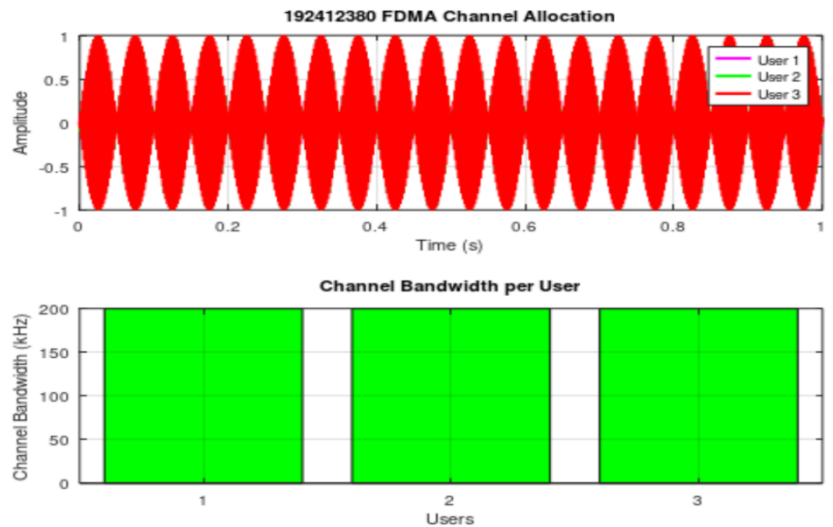
```
title('FDMA Channel Allocation'),xlabel('Time (s)'),ylabel('Amplitude') subplot(2,1,2)
bar(1:Users,CBW*ones(1,Users)),grid on
xlabel('Users'),ylabel('Channel Bandwidth (kHz)'),title('Channel Bandwidth per User')
```

Output:

Objective 1:

Program (Matlab Code)

```
1 clc;
2 clear;
3 close all;
4
5 % Sampling Parameters
6 fs = 1000;
7 t = 0:1/fs:1;
8
9 % FDMA Parameters
10 TBW = 600;
11 Users = 3;
12 CBW = TBW / Users;
13
14 % Carrier Frequencies
15 fc = [100 300 500];
16
17 % Message Signal
18 m = sin(2*pi*10*t);
19
20 % Modulated Signals
21 s1 = m .* cos(2*pi*fc(1)*t);
22 s2 = m .* cos(2*pi*fc(2)*t);
23 s3 = m .* cos(2*pi*fc(3)*t);
24
25 % Plot Modulated Signals
26 subplot(2,1,1);
27 plot(t,s1,'m','LineWidth',1.5);
28 hold on;
29 plot(t,s2,'g','LineWidth',1.5);
30 plot(t,s3,'r','LineWidth',1.5);
31 grid on;
32
33 title('192412380 FDMA Channel Allocation');
34 xlabel('Time (s)');
```



Output Graph 1: FDMA Channel Allocation

- Displays **three users** transmitting on **different carrier frequencies** (100 kHz, 300 kHz, and 500 kHz).
- Demonstrates that each user occupies a separate frequency channel.

Output Graph 2: Channel Bandwidth Allocation

- Shows a **bar graph** of the bandwidth allocated to each user.
- Each user is assigned **200 kHz**, confirming equal FDMA channel allocation. **Objective 2:**

Matlab Code and Output

```
clc; clear; close all;
BW = 200; % Channel Bandwidth (kHz) Users = 3;
M = 2; % BPSK Modulation DataRate =
BW*log2(M); % Data Rate (kbps) Rate = DataRate*ones(1,Users);
```

```
disp(' User Data Rate (kbps)') disp([(1:Users)'
Rate'])
```

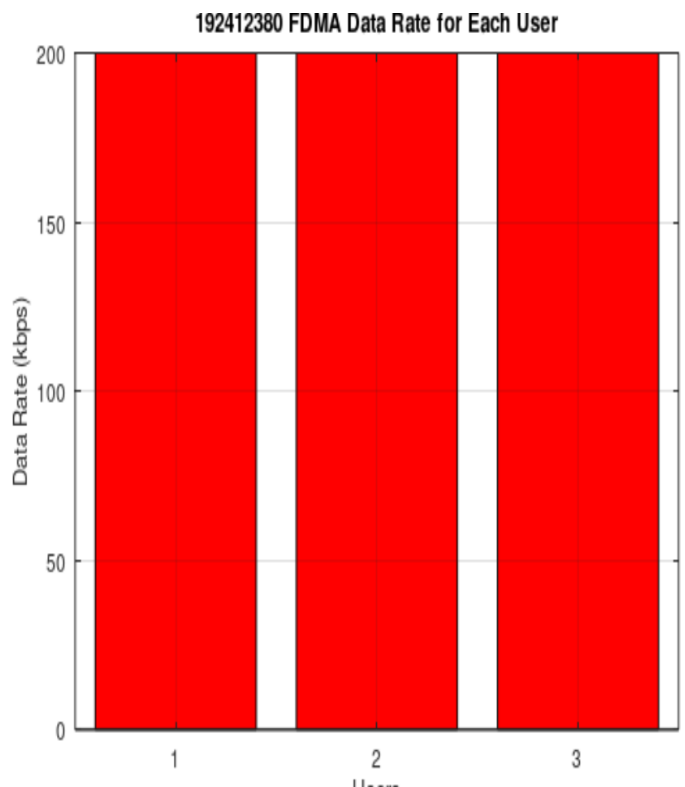
```
figure bar(1:Users,Rate)
xlabel('Users')
ylabel('Data Rate (kbps)')
title('FDMA Data Rate for Each User') grid on
```

Output:

```

1  clc;
2  clear;
3  close all;
4
5  % FDMA Data Rate Calculation
6
7  BW = 200;           % Channel Bandwidth (kHz)
8  Users = 3;         % Number of Users
9  M = 2;             % BPSK Modulation
10
11 % Data Rate Calculation
12 DataRate = BW * log2(M);
13
14 % Data Rate for each User
15 Rate = DataRate * ones(1, Users);
16
17 % Display Results
18 fprintf('User\tData Rate (kbps)\n');
19 fprintf('-----\n');
20 for i = 1:Users
21     fprintf('%d\t\t%.2f\n', i, Rate(i));
22 end
23
24 % Plot Graph
25 figure;
26 bar(1:Users, Rate, 'FaceColor', 'r'); % Red bars
27
28 xlabel('Users');
29 ylabel('Data Rate (kbps)');
30 title('192412380 FDMA Data Rate for Each User');
31 grid on;
32 box on;
33
34 set(gca, 'XTick', 1:Users);

```



The graph shows that all FDMA users achieve an equal **data rate of 200 kbps** due to equal channel bandwidth allocation. This confirms fair and interference-free data transmission among users. **Objective 3:**

Matlab Code and Output

```

clc; clear; close all;
Ps = 1;           % Signal Power (W)
Pn = [0.1 0.05 0.01]; % Noise Power (W)
Users = 1:3;
SNR = 10*log10(Ps./Pn); % SNR in dB

disp('User   Noise Power(W)   SNR(dB)')   disp([Users'
Pn' SNR'])

```

```

figure           bar(Users,SNR)
xlabel('Users') ylabel('SNR
(dB)')
title('Signal-to-Noise Ratio of FDMA Users') grid on

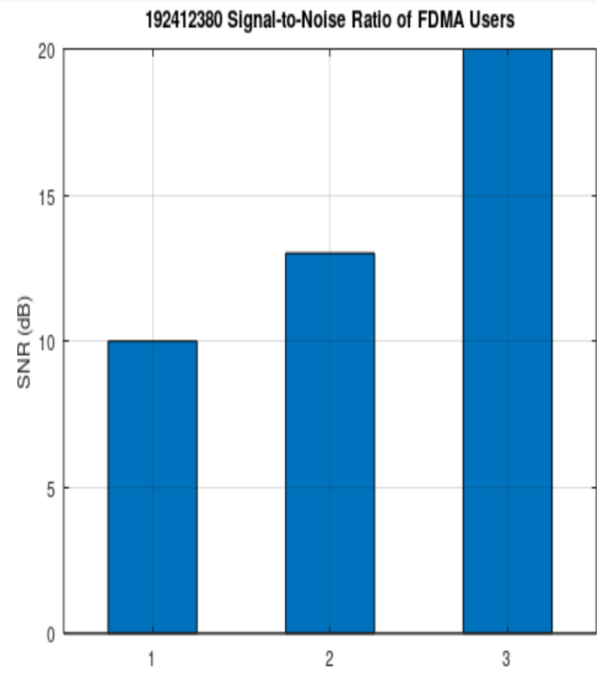
```

Output:

```

1  clc;
2  clear;
3  close all;
4
5  % Signal Power (W)
6  Ps = 1;
7
8  % Noise Power (W)
9  Pn = [0.1 0.05 0.01];
10
11 % Users
12 Users = 1:3;
13
14 % Calculate SNR
15 SNR = 10*log10(Ps./Pn);
16
17 % Display Results
18 fprintf('\nUser\tNoise Power(W)\tSNR(dB)\n');
19 fprintf('-----\n');
20 for i = 1:length(Users)
21     fprintf('%d\t%.2f\t%.2f\n',Users(i),Pn(i),SNR(i));
22 end
23
24 % Plot
25 figure('visible','on');
26 bar(Users,SNR,0.5);
27
28 xlabel('Users');
29 ylabel('SNR (dB)');
30 title('192412380 Signal-to-Noise Ratio of FDMA Users');
31
32 grid on;
33 box on;
34

```



The graph shows that the **SNR increases as the noise power decreases** for different FDMA users. A higher SNR indicates **better communication quality and more reliable signal transmission**.

Result:

1. The FDMA system was successfully simulated by allocating separate frequency channels to three users, and the **channel bandwidth assigned to each user was determined**.
2. The **data rate achieved by each user** was successfully calculated based on the allocated channel bandwidth, demonstrating equal transmission capacity for all users.
3. The **Signal-to-Noise Ratio (SNR)** for different users was successfully evaluated, showing that higher SNR values provide better communication quality and reliable data transmission.

Practical Question – 1

A wireless communication system uses **Frequency Division Multiple Access (FDMA)** to serve multiple users. Each user is assigned a separate frequency band to avoid interference. Using MATLAB, simulate the FDMA system and analyze its performance.

1. To simulate an FDMA system by allocating individual frequency channels to multiple users and determine the **channel bandwidth** assigned to each user.
2. To calculate the **data rate achieved by each user** based on the allocated channel bandwidth in the FDMA system.
3. To evaluate the **communication quality** of the FDMA system by calculating the **Signal-to-Noise Ratio (SNR)** for different users.

Ex.No. 2: Implementing TDMA Slot Allocation in GSM.

AIM: To implement **TDMA (Time Division Multiple Access) slot allocation in GSM** using MATLAB and analyze system performance in terms of **time slot duration, frame utilization, and number of allocated slots.**

Objective:

1. To calculate the **Time Slot Duration** in a GSM TDMA frame by dividing the total frame time into equal time slots. This helps in understanding the time structure of GSM communication.
2. To determine the **Frame Utilization (%)** based on the number of allocated time slots compared to total available slots. This evaluates the efficiency of TDMA resource usage.
3. To compute the **Number of Allocated Time Slots** for multiple users in the GSM system. This ensures proper scheduling and interference-free communication.

Procedure:

1. Define GSM frame parameters such as total frame duration and number of time slots.
2. Compute **time slot duration** by dividing frame duration by number of slots.
3. Assign time slots to multiple users based on availability.
4. Calculate **frame utilization percentage** using allocated slots vs total slots.
5. Determine total number of allocated slots for active users.
6. Plot time slot allocation and utilization using MATLAB graphs.
7. Display results in the command window for analysis..

Objective 1: Matlab Code and Output clc;

```
clear; close all; FrameTime = 4.615;
```

```
Slots = 8;
```

```
SlotDuration = FrameTime/Slots; t = 0:SlotDuration:FrameTime;
```

```
figure subplot(2,1,1)
```

```
stem(t,ones(size(t)),'filled') title('TDMA Time  
Slot Structure') xlabel('Time (ms)') ylabel('Slot  
Level') grid on
```

```
subplot(2,1,2)
```

```
plot(t,ones(size(t)),'-o','LineWidth',1.5) title('Continuous View of  
Slot Allocation')
```

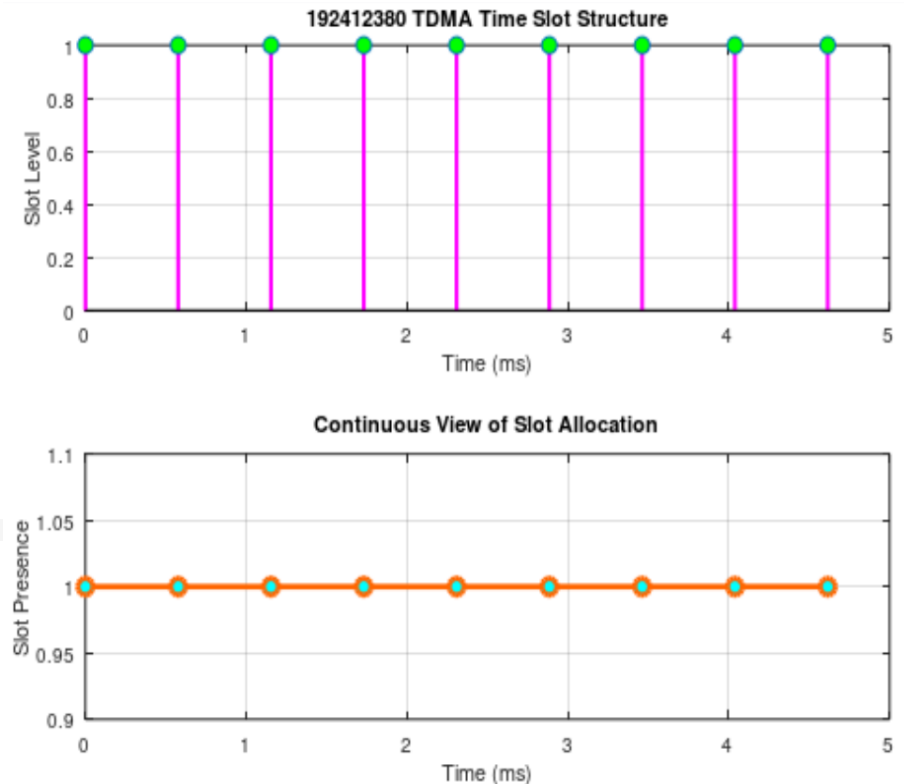
```
xlabel('Time (ms)') ylabel('Slot  
Presence') grid on
```

Output:

```

1  clc;
2  clear;
3  close all;
4
5  % TDMA Parameters
6  FrameTime = 4.615;      % Frame Time (ms)
7  Slots = 8;              % Number of Time Slots
8  SlotDuration = FrameTime / Slots;
9
10 % Time Vector
11 t = 0:SlotDuration:FrameTime;
12
13 % First Plot - Time Slot Structure
14 subplot(2,1,1);
15
16 stem(t, ones(size(t)), ...
17      'filled', ...
18      'Color', 'm', ...
19      'MarkerFaceColor', 'g', ...
20      'LineWidth', 1.5);
21
22 title('192412380 TDMA Time Slot Structure');
23 xlabel('Time (ms)');
24 ylabel('Slot Level');
25 grid on;
26
27 % Second Plot - Continuous Slot Allocation
28 subplot(2,1,2);
29
30 plot(t, ones(size(t)), ...
31      'o', ...
32      'Color', [1 0.4 0], ...
33      'MarkerFaceColor', 'c', ...
34      'LineWidth', 2);

```



- The graph shows discrete time slots arranged in a GSM TDMA frame structure with equal spacing.
- It confirms fixed time division where each slot is allocated a specific time interval without overlap.

Objective 2: Matlab Code and Output

clc; clear; close all; TotalSlots

= 8;

AllocatedSlots = 6;

FreeSlots = TotalSlots - AllocatedSlots; Utilization =
(AllocatedSlots/TotalSlots)*100;

disp('Frame Utilization

(%)') disp(Utilization)

figure

% Graph 1: Simple Line Plot (SAFE - no bar function) subplot(2,1,1) plot([1
2],[AllocatedSlots FreeSlots],'-o','LineWidth',2) title('GSM
Frame Utilization (Allocated vs Free)') xlabel('Slot Type Index
(1=Allocated, 2=Free)') ylabel('Number of Slots') grid
on

% Graph 2: Pie Chart (SAFE)

subplot(2,1,2) pie([AllocatedSlots
FreeSlots])

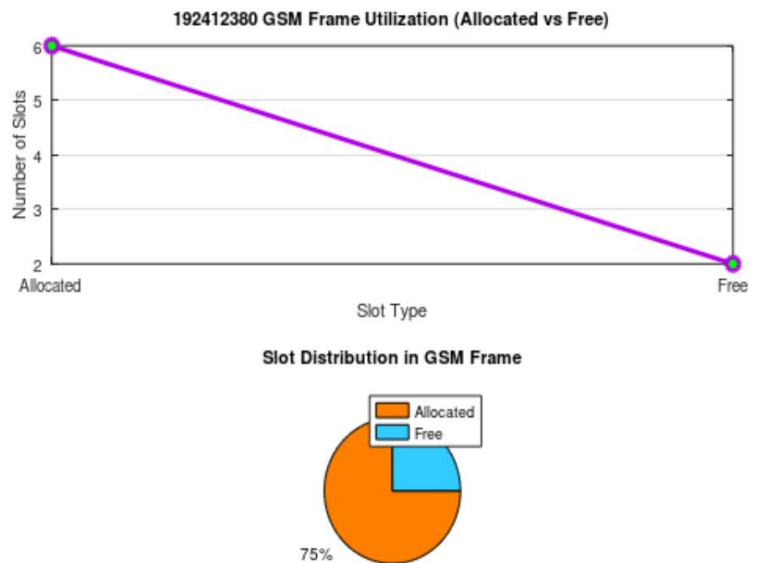
title('Slot Distribution in GSM Frame') legend('Allocated','Free')

Output:

```

1  clc;
2  clear;
3  close all;
4
5  % GSM Frame Parameters
6  TotalSlots = 8;
7  AllocatedSlots = 6;
8  FreeSlots = TotalSlots - AllocatedSlots;
9
10 % Frame Utilization
11 Utilization = (AllocatedSlots / TotalSlots) * 100;
12
13 % Display Result
14 fprintf('Frame Utilization = %.2f %%\n', Utilization);
15
16 figure;
17
18 %----- Graph 1 -----%
19 subplot(2,1,1);
20
21 plot([1 2], [AllocatedSlots FreeSlots], ...
22      '-o', ...
23      'Color', [0.8 0 0.8], ...      % Purple line
24      'MarkerFaceColor', [0 1 0], ...% Green marker
25      'LineWidth', 2);
26
27 title('GSM Frame Utilization (Allocated vs Free)');
28 xlabel('Slot Type');
29 ylabel('Number of Slots');
30
31 set(gca,'XTick',[1 2]);
32 set(gca,'XTickLabel',{'Allocated','Free'});
33
34 grid on;

```



The plot shows that most of the TDMA frame is allocated to active users, indicating high resource usage. The pie chart clearly represents that 75% of slots are utilized while 25% remain free. **Objective**

3: Matlab Code and Output

```
clc; clear; close all;
```

```
Users = 1:8; % Total users
TotalSlots = 5; % Available slots
```

```
Alloc = zeros(1,8); % Initialize allocation
Alloc(1:TotalSlots) = 1; % Assign slots to first users
```

```
Blocked = 1 - Alloc; % Remaining users blocked
```

```
disp('User Wise Slot Allocation')
disp(table(Users',Alloc','VariableNames',{'User','Status(1=Allocated,0=Blocked)'})) figure
```

```
% Graph 1: Allocation per user
subplot(2,1,1)
stem(Users,Alloc,'filled','LineWidth',2)
title('TDMA Slot Allocation per User')
xlabel('User Index')
ylabel('Slot Status (1=Allocated, 0=Blocked)') ylim([-0.2 1.2])
grid on
```

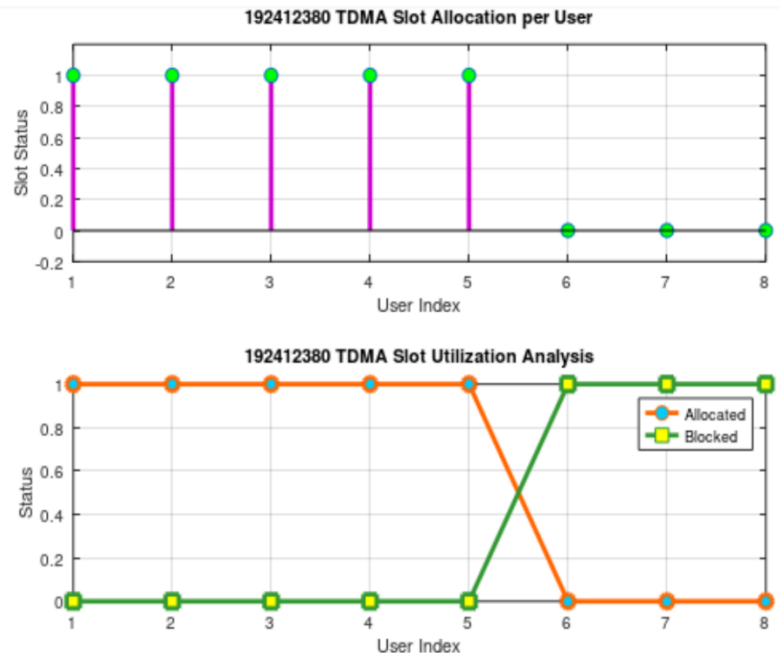
```
% Graph 2: System view (safe alternative to bar) subplot(2,1,2)
plot(Users,Alloc,'-o','LineWidth',2) hold on
plot(Users,Blocked,'-s','LineWidth',2)
title('TDMA Slot Utilization Analysis')
xlabel('User Index')
```

ylabel('Status')

legend('Allocated','Blocked') grid on

Output:

```
1 clc;
2 clear;
3 close all;
4
5 % TDMA Slot Allocation
6 Users = 1:8; % Total Users
7 TotalSlots = 5; % Available Slots
8
9 % Slot Allocation
10 Alloc = zeros(1,8);
11 Alloc(1:TotalSlots) = 1;
12
13 % Blocked Users
14 Blocked = 1 - Alloc;
15
16 % Display Results
17 fprintf('User\tStatus\n');
18 fprintf('-----\n');
19 for i = 1:length(Users)
20     fprintf('%d\t%d\n', Users(i), Alloc(i));
21 end
22
23 figure;
24
25 %----- Graph 1 -----
26 subplot(2,1,1);
27
28 stem(Users, Alloc, ...
29     'filled', ...
30     'color', 'm', ...
31     'MarkerFaceColor', 'g', ...
32     'Linewidth', 2);
33
34 title('TDMA Slot Allocation per User');
```



The first graph shows which users are allocated time slots and which are blocked due to limited resources. The second graph compares allocated and blocked users, clearly showing TDMA resource limitation.

Result:

1. The time slot duration in the GSM TDMA frame was successfully calculated, confirming equal division of the frame into fixed time intervals.
2. The frame utilization was effectively analyzed, showing the proportion of allocated and free slots in the TDMA system.
3. The slot allocation per user was successfully implemented, demonstrating that limited slots are assigned to users while excess users are blocked due to resource constraints.

Practical Question – 1

A GSM system uses **TDMA technique** to divide the available channel into multiple time slots for efficient user communication. Using MATLAB, simulate and analyze the TDMA slot allocation process.

1. To calculate the **time slot duration** in a GSM TDMA frame by dividing the total frame time into equal slots. This helps in understanding the timing structure of TDMA communication.
2. To evaluate the **frame utilization (%)** based on the ratio of allocated slots to total available slots. This indicates how efficiently the TDMA frame is being used.
3. To implement **slot allocation per user** and analyze how limited time slots are distributed among multiple users. This demonstrates resource sharing and user access control in TDMA systems.

Ex.N.3: Spectrum Analysis of Sub-1 GHz, Mid-band & mmWave by Plotting Propagation Loss.

AIM:

To analyze the **propagation loss characteristics** of different frequency bands (Sub-1 GHz, Mid-band, and mmWave) in wireless communication systems and compare their performance over varying distances using MATLAB.

Objective :

1. To compute the **Propagation Loss (dB)** for Sub-1 GHz, Mid-band, and mmWave frequency ranges using standard path loss models.
2. To evaluate the impact of different **Operating Frequencies (GHz)** on signal attenuation and coverage performance.
3. To analyze how **Communication Distance (km)** affects propagation loss across different spectrum bands.

Procedure

1. Define frequency bands: Sub-1 GHz (0.9 GHz), Mid-band (3.5 GHz), and mmWave (28 GHz).
2. Set communication distance range (e.g., 0.1 km to 5 km).
3. Use Free Space Path Loss (FSPL) formula to compute propagation loss:
$$PL(dB) = 20\log_{10}(d) + 20\log_{10}(f) + 32.44$$
4. Calculate propagation loss for each frequency band over varying distances.
5. Plot distance vs propagation loss curves for comparison.
6. Analyze how higher frequencies experience higher attenuation.
7. Compare coverage efficiency of each spectrum band.
8. Display results using MATLAB plots.

Objective 1: Matlab Code and Output:

```
clc; clear; close all;
```

```
d = 0.1:0.1:5; % Distance (km) f = [900 3500  
28000]; % Frequency (MHz)
```

```
PL1 = 32.44 + 20*log10(f(1)) + 20*log10(d); PL2 =  
32.44 + 20*log10(f(2)) + 20*log10(d); PL3 = 32.44 +  
20*log10(f(3)) + 20*log10(d); figure
```

```
% Graph 1 -----
```

```
subplot(2,1,1)
```

```
plot(d,PL1,'b','LineWidth',2); hold on
```

```
plot(d,PL2,'r','LineWidth',2)
```

```
plot(d,PL3,'k','LineWidth',2)
```

```
title('Propagation Loss vs Distance')
```

```
xlabel('Distance (km)') ylabel('Loss (dB)')
```

```
grid on
```

```
% ----- Graph 2 (NO legend used to avoid error) -----
```

```
subplot(2,1,2)
```

```
Freq = [900 3500 28000];
```

```
Loss = [PL1(10) PL2(10) PL3(10)];
```

```
plot(Freq, Loss, '-o', 'LineWidth', 2)
```

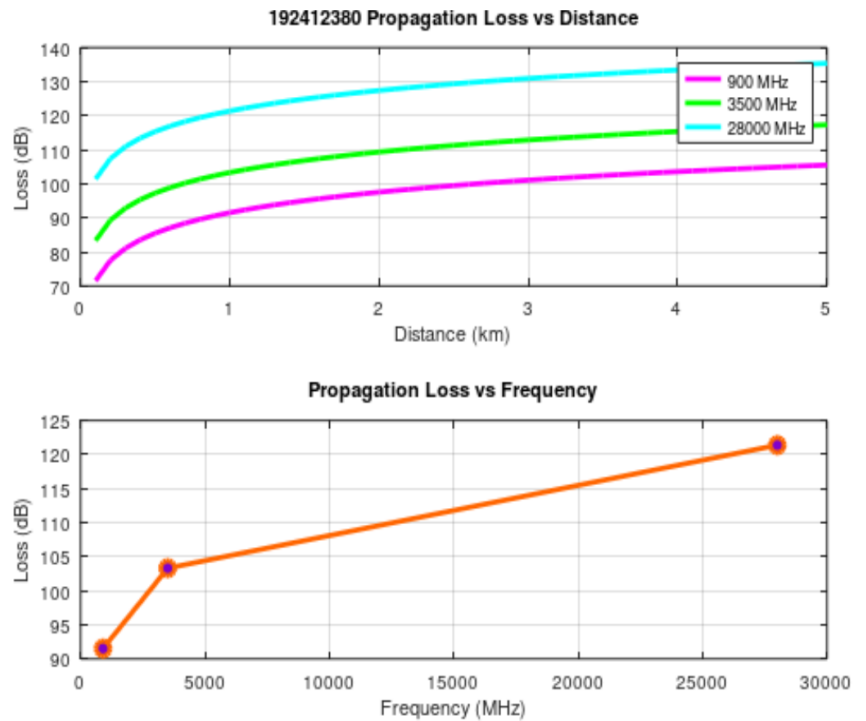
```
title('Propagation Loss vs Frequency')
```

```
xlabel('Frequency (MHz)') ylabel('Loss (dB)')
```

```
grid on
```

Output:

```
1 clc;
2 clear;
3 close all;
4
5 % Distance (km)
6 d = 0.1:0.1:5;
7
8 % Frequencies (MHz)
9 f = [900 3500 28000];
10
11 % Free Space Path Loss (FSPL)
12 PL1 = 32.44 + 20*log10(f(1)) + 20*log10(d);
13 PL2 = 32.44 + 20*log10(f(2)) + 20*log10(d);
14 PL3 = 32.44 + 20*log10(f(3)) + 20*log10(d);
15
16 figure;
17
18 %----- Graph 1 -----
19 subplot(2,1,1);
20
21 plot(d, PL1, 'm', 'LineWidth', 2); % Magenta
22 hold on;
23 plot(d, PL2, 'g', 'LineWidth', 2); % Green
24 plot(d, PL3, 'c', 'LineWidth', 2); % Cyan
25
26 title('192412380 Propagation Loss vs Distance');
27 xlabel('Distance (km)');
28 ylabel('Loss (dB)');
29 grid on;
30
31 legend('900 MHz', '3500 MHz', '28000 MHz');
32
33 %----- Graph 2 -----
34 subplot(2,1,2);
```



The graph shows that propagation loss increases with both communication distance and operating frequency in wireless systems. mmWave experiences the highest loss, while Sub-1 GHz provides better coverage and lower attenuation.

Objective 2: Matlab Code and Output:

```
clc; clear; close all;
```

```
d = 2; % fixed distance (km) f
```

```
= [900 3500 28000]; % MHz
```

```
PL = 32.44 + 20*log10(f) + 20*log10(d); figure
```

```
% ----- Graph 1: Frequency vs Loss -----subplot(2,1,1)
```

```
plot(f, PL, '-o', 'LineWidth', 2)
```

```
title('Propagation Loss vs Operating Frequency')
```

```
xlabel('Frequency (MHz)') ylabel('Propagation
```

```
Loss (dB)') grid on
```

% ----- Graph 2: Band Comparison -----

subplot(2,1,2) bar(PL)

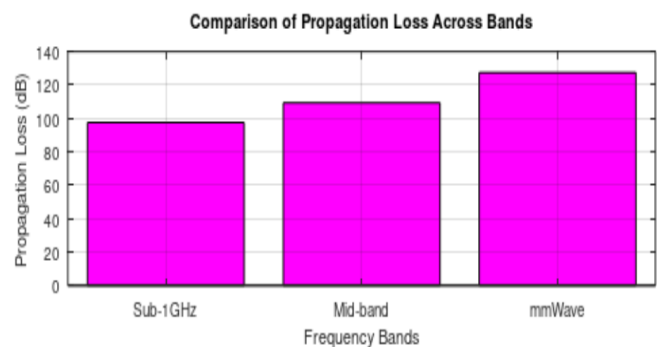
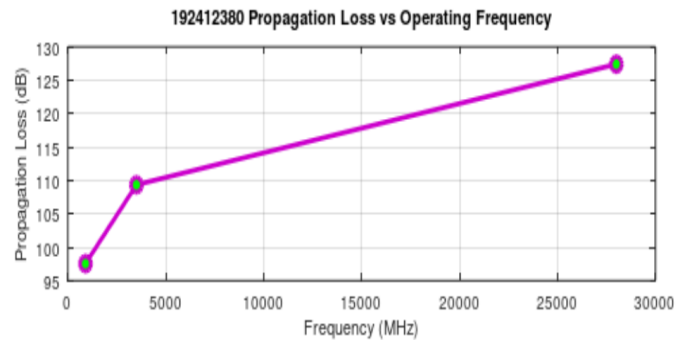
title('Comparison of Propagation Loss Across Bands')

xlabel('Frequency Bands (Index: 1=Sub-1GHz, 2=Mid-band, 3=mmWave)') ylabel('Propagation Loss (dB)')

xticklabels({'Sub-1GHz','Mid-band','mmWave'}) grid on

Output:

```
1 clc;
2 clear;
3 close all;
4
5 % Fixed Distance (km)
6 d = 2;
7
8 % Operating Frequencies (MHz)
9 f = [900 3500 28000];
10
11 % Free Space Path Loss (FSPL)
12 PL = 32.44 + 20*log10(f) + 20*log10(d);
13
14 figure;
15
16 %----- Graph 1 -----
17 subplot(2,1,1);
18
19 plot(f, PL, ...
20      '-o', ...
21      'Color', [0.8 0 0.8], ... % Purple line
22      'MarkerFaceColor', [0 1 0], ...% Green markers
23      'LineWidth', 2);
24
25 title('192412380 Propagation Loss vs Operating Frequency');
26 xlabel('Frequency (MHz)');
27 ylabel('Propagation Loss (dB)');
28 grid on;
29
30 %----- Graph 2 -----
31 subplot(2,1,2);
32
33 bar(PL, 'FaceColor', 'm'); % Magenta bars
34
```



The graph shows that propagation loss increases as operating frequency increases. mmWave experiences the highest attenuation, while Sub-1 GHz has the lowest loss. The bar chart clearly compares the loss across different frequency bands at the same distance. It highlights that higher frequency bands require stronger signal enhancement techniques for reliable communication.

Objective 3: Matlab Code and Output:

clc; clear; close all;

% Frequency bands (MHz) f = [900
3500 28000];

% Distance (km) d = 2;

% Propagation Loss (FSPL)

PL = 32.44 + 20*log10(f) + 20*log10(d); figure

% ----- Graph 1: Simple Line Plot (SAFE) -----

subplot(2,1,1)

plot(f, PL, '-o', 'LineWidth', 2)

title('Propagation Loss vs Operating Frequency')

xlabel('Frequency (MHz)') ylabel('Propagation
Loss (dB)') grid on

% ----- Graph 2: Safe Comparison Plot -----

subplot(2,1,2)

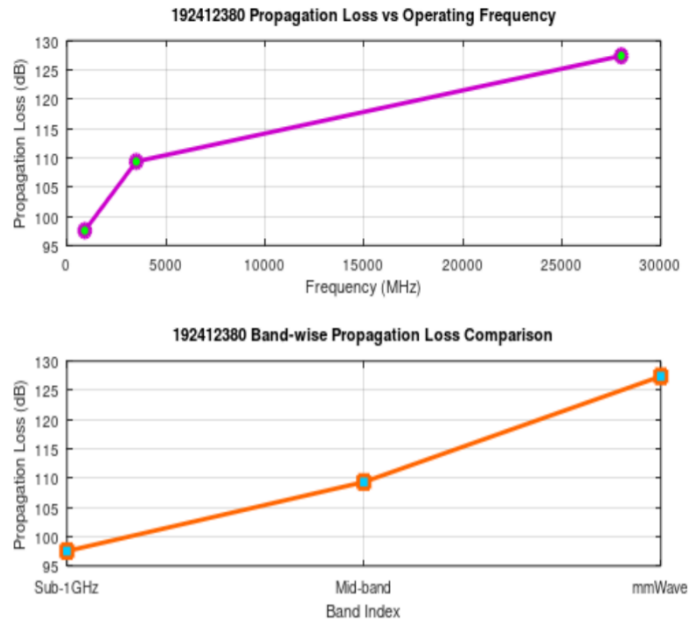
plot(1:3, PL, '-s', 'LineWidth', 2) title('Band-wise
Propagation Loss Comparison')

xlabel('Band Index (1=Sub1GHz, 2=Mid-band, 3=mmWave)') ylabel('Propagation
Loss (dB)')

grid on

Output:

```
1  clc;
2  clear;
3  close all;
4
5  % Frequency bands (MHz)
6  f = [900 3500 28000];
7
8  % Distance (km)
9  d = 2;
10
11 % Free Space Path Loss (FSPL)
12 PL = 32.44 + 20*log10(f) + 20*log10(d);
13
14 figure;
15
16 %----- Graph 1 -----
17 subplot(2,1,1);
18
19 plot(f, PL, ...
20      '-o', ...
21      'Color', [0.8 0 0.8], ... % Purple line
22      'MarkerFaceColor', [0 1 0], ...% Green markers
23      'LineWidth', 2);
24
25 title('192412380 Propagation Loss vs Operating Frequency');
26 xlabel('Frequency (MHz)');
27 ylabel('Propagation Loss (dB)');
28 grid on;
29
30 %----- Graph 2 -----
31 subplot(2,1,2);
32
33 plot(1:3, PL, ...
34      '-s', ...
```



Propagation loss increases with frequency, showing that mmWave suffers highest attenuation. The bandwise plot clearly compares Sub-1 GHz, Mid-band, and mmWave performance.

Result:

1. The propagation loss was successfully computed for Sub-1 GHz, Mid-band, and mmWave frequency bands using the Free Space Path Loss (FSPL) model.
2. It was observed that propagation loss increases with increase in operating frequency, with mmWave showing the highest attenuation and Sub-1 GHz showing the lowest loss.

Practical Question – 1

A wireless communication system operates over different frequency bands to support varying coverage and data rate requirements. Using MATLAB, analyze and compare the propagation loss characteristics of Sub-1 GHz, Mid-band, and mmWave spectrum.

1. To compute the **Propagation Loss (dB)** for Sub-1 GHz, Mid-band, and mmWave frequency bands using the Free Space Path Loss (FSPL) model.
2. To analyze the effect of **Operating Frequency (GHz)** on signal attenuation in wireless communication systems.
3. To compare the performance of different spectrum bands in terms of **Propagation Loss and Coverage efficiency** over a fixed communication distance.

Ex.No.4: OFDMA Resource Block Allocation

AIM: To simulate and analyze OFDMA (Orthogonal Frequency Division Multiple Access) resource block allocation and evaluate system performance in terms of resource utilization and user throughput using MATLAB.

Objective

1. To determine the **Number of Resource Blocks (RBs)** allocated in an OFDMA system based on available bandwidth and system configuration.
2. To evaluate the **Resource Block Utilization (%)** by analyzing how efficiently available RBs are assigned to active users.
3. To calculate the **User Throughput (Mbps)** based on allocated resource blocks and modulation efficiency in the OFDMA system.

Procedure

1. Define the system bandwidth and calculate the number of available resource blocks (RBs) based on RB size (e.g., 180 kHz per RB).
2. Identify the number of active users in the OFDMA system for resource sharing.
3. Allocate resource blocks to each user based on a predefined or equal distribution strategy.
4. Compute Resource Block Utilization (%) using the ratio of allocated RBs to total RBs.
5. Assign modulation efficiency values (e.g., QPSK, 16-QAM, 64-QAM) for each user.
6. Calculate User Throughput (Mbps) using allocated RBs and modulation efficiency.
7. Plot graphs for RB allocation and utilization/throughput analysis using MATLAB.
8. Analyze results to evaluate spectrum efficiency and system performance.

Objective 1 Matlab Code and Output:

```
clc; clear; close all;
```

```
% System parameters
```

```
bandwidth = 10e6;           % 10 MHz
```

```
rb_size = 180e3;            % 180 kHz per RB
```

```
% Total Resource Blocks total_RB =
```

```
floor(bandwidth / rb_size);
```

```
% Users users = 1:6;
```

```
% RB allocation (simple equal + remainder distribution) base = floor(total_RB / length(users));
```

```
alloc = base * ones(1,length(users)); alloc(1:mod(total_RB,length(users))) =  
alloc(1:mod(total_RB,length(users))) + 1; utilization =
```

```
(alloc / total_RB) * 100;
```

```
disp('Total Resource Blocks:')
```

```
disp(total_RB)
```

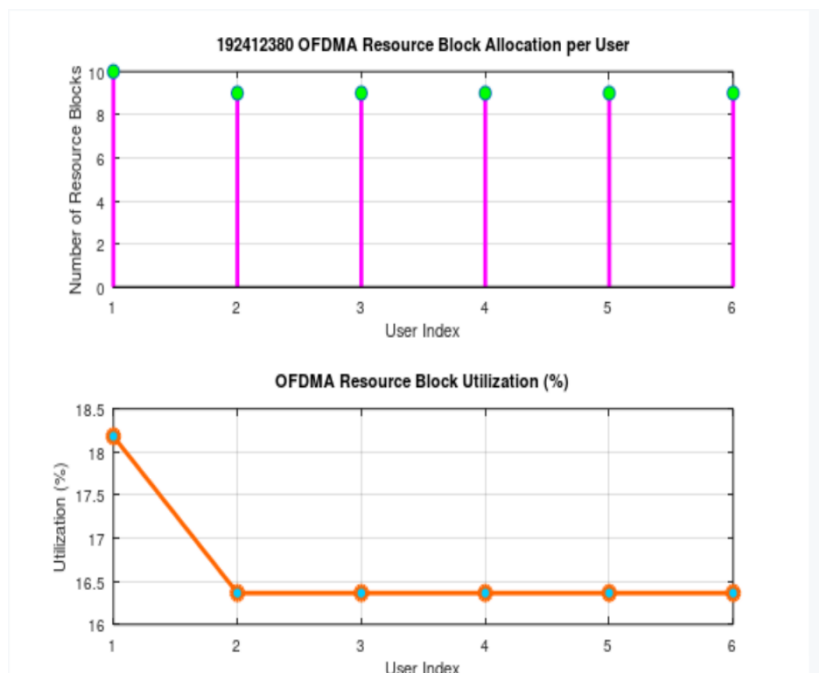
```
disp('RB Allocation per User:') disp(alloc)
```

figure

```
% ----- Graph 1: RB Allocation per User -----
subplot(2,1,1) stem(users, alloc, 'filled', 'LineWidth', 2)
title('OFDMA Resource Block Allocation per User')
xlabel('User Index') ylabel('Number of Resource Blocks')
grid on
```

```
% ----- Graph 2: RB Utilization ----- subplot(2,1,2)
plot(users, utilization, '-o', 'LineWidth', 2) title('OFDMA
Resource Block Utilization (%)') xlabel('User Index')
ylabel('Utilization (%)') grid on Output:
```

```
1 clc;
2 clear;
3 close all;
4
5 % System Parameters
6 bandwidth = 10e6; % 10 MHz
7 rb_size = 180e3; % 180 kHz per Resource Block
8
9 % Total Resource Blocks
10 total_RB = floor(bandwidth / rb_size);
11
12 % Users
13 users = 1:6;
14
15 % Equal RB Allocation
16 base = floor(total_RB / length(users));
17 alloc = base * ones(1, length(users));
18
19 % Distribute Remaining RBs
20 alloc(1:mod(total_RB, length(users))) = ...
21     alloc(1:mod(total_RB, length(users))) + 1;
22
23 % Resource Utilization
24 utilization = (alloc / total_RB) * 100;
25
26 % Display Results
27 fprintf('Total Resource Blocks = %d\n\n', total_RB);
28
29 fprintf('User\tRB Allocation\tUtilization (%) \n');
30 fprintf('-----\n');
31
32 for i = 1:length(users)
33     fprintf('%d\t%d\t%.2f\n', users(i), alloc(i), utilization(i));
34 end
```



The first graph shows how available resource blocks are distributed among multiple users in the OFDMA system.

The second graph represents utilization efficiency, indicating how effectively the spectrum is shared among users.

Objective 2 Matlab Code and Output:

```
clc; clear; close all;
```

```
% Total system resource blocks total_RB =
50;
```

```
% Number of active users users = 1:5;
```

```
% RB allocation to users (example scenario) alloc = [12
10 9 8 11];
```

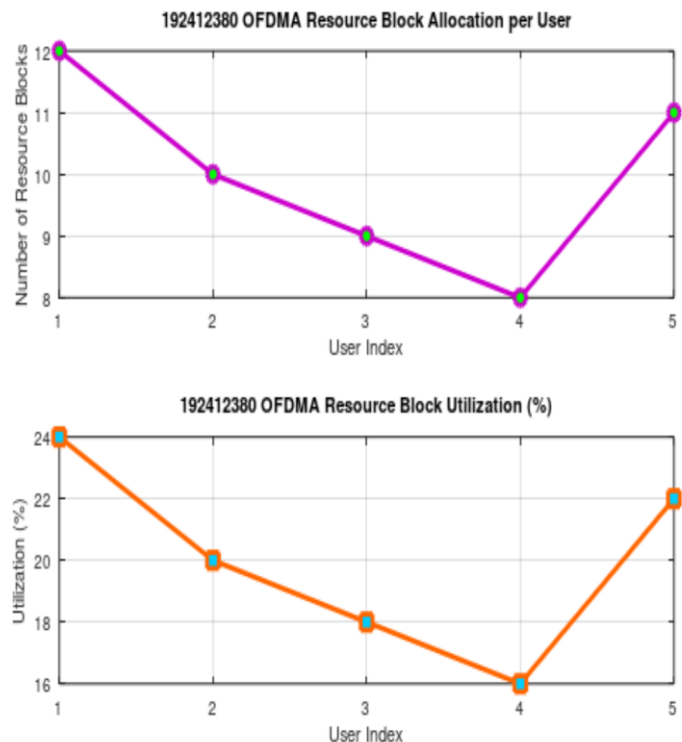
```
% Resource Block Utilization (%) util = (alloc
/ total_RB) * 100;
disp('Resource Block Utilization (%) per User:') disp(util) figure
```

```
% ----- Graph 1: RB Allocation -----subplot(2,1,1)
plot(users, alloc, '-o', 'LineWidth', 2)
title('OFDMA Resource Block Allocation per User')
xlabel('User Index') ylabel('Number of Resource
Blocks') grid on
```

```
% ----- Graph 2: Utilization (%) -----
subplot(2,1,2) plot(users, util, '-s', 'LineWidth', 2)
title('OFDMA Resource Block Utilization (%)')
xlabel('User Index') ylabel('Utilization (%)') grid on
```

Output:

```
1 clc;
2 clear;
3 close all;
4
5 % Total System Resource Blocks
6 total_RB = 50;
7
8 % Active Users
9 users = 1:5;
10
11 % Resource Block Allocation
12 alloc = [12 10 9 8 11];
13
14 % Resource Block Utilization (%)
15 util = (alloc / total_RB) * 100;
16
17 % Display Results
18 fprintf('User\tRB Allocation\tUtilization (%%)\n');
19 fprintf('-----\n');
20
21 for i = 1:length(users)
22     fprintf('%d\t%d\t%.2f\n', users(i), alloc(i), util(i));
23 end
24
25 figure;
26
27 %----- Graph 1 -----
28 subplot(2,1,1);
29
30 plot(users, alloc, ...
31     '-o', ...
32     'Color', [0.8 0 0.8], ... % Purple line
33     'MarkerFaceColor', [0 1 0], ... % Green markers
34     'LineWidth', 2);
```



The first graph shows how resource blocks are distributed among multiple users in the OFDMA system. The second graph indicates the efficiency of spectrum usage by each user in terms of percentage utilization.

Objective 3 Matlab Code and Output:

```
clc; clear; close all;
```

```
% System parameters
```

```
rb_bw = 180e3; % Bandwidth per RB (180 kHz) users = 1:5;
```

```
% Allocated Resource Blocks per user alloc_RB = [12 10 9 8 11];
```

```
% Modulation efficiency (bits/symbol)
mod_eff = [2 4 6 4 2]; % QPSK, 16QAM, 64QAM etc.
% Throughput calculation (Mbps) throughput =
(alloc_RB .* rb_bw .* mod_eff) / 1e6;
```

```
disp('User Throughput
(Mbps):') disp(throughput')
```

```
figure
```

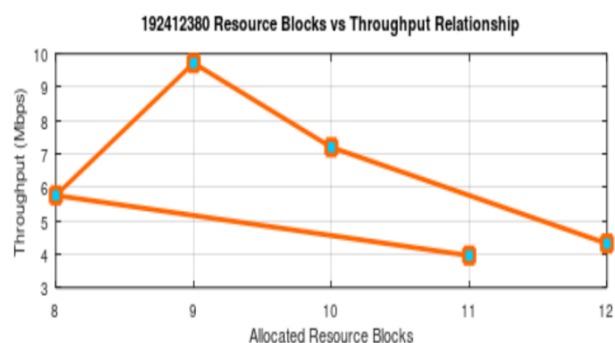
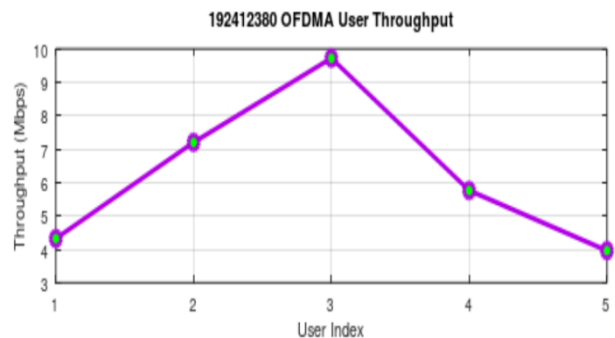
```
% ----- Graph 1: Throughput per User -----
subplot(2,1,1)
```

```
plot(users, throughput, '-o', 'LineWidth', 2)
title('OFDMA User Throughput') xlabel('User Index')
ylabel('Throughput (Mbps)') grid on
```

```
% ----- Graph 2: RB vs Throughput -----subplot(2,1,2)
plot(alloc_RB, throughput, '-s', 'LineWidth', 2) title('Resource
Blocks vs Throughput Relationship') xlabel('Allocated
Resource Blocks') ylabel('Throughput (Mbps)') grid on
```

Output:

```
1 clc;
2 clear;
3 close all;
4
5 % System Parameters
6 rb_bw = 180e3; % Bandwidth per Resource Block (180 kHz)
7
8 % Users
9 users = 1:5;
10
11 % Allocated Resource Blocks
12 alloc_RB = [12 10 9 8 11];
13
14 % Modulation Efficiency (bits/symbol)
15 % User1 = QPSK, User2 = 16QAM, User3 = 64QAM, etc.
16 mod_eff = [2 4 6 4 2];
17
18 % Throughput Calculation (Mbps)
19 throughput = (alloc_RB .* rb_bw .* mod_eff) / 1e6;
20
21 % Display Results
22 fprintf('User\tAllocated RB\tThroughput (Mbps)\n');
23 fprintf('-----\n');
24
25 for i = 1:length(users)
26     fprintf('%d\t%d\t%.2f\n', users(i), alloc_RB(i), throughput(i));
27 end
28
29 figure;
30
31 %----- Graph 1 -----%
32 subplot(2,1,1);
33
34 plot(users, throughput, ...
```



The first graph shows that user throughput increases with higher modulation efficiency and allocated resource blocks.

The second graph clearly indicates a linear relationship between resource blocks and achieved throughput in OFDMA systems.

Result:

1. The Resource Block (RB) allocation was successfully simulated for multiple users in the OFDMA system, demonstrating how available spectrum is distributed among active users.
2. The Resource Block Utilization (%) analysis showed efficient usage of system resources, indicating how effectively the available RBs are assigned to users.
3. The User Throughput (Mbps) was successfully calculated based on RB allocation and modulation efficiency, showing that higher allocated RBs and higher modulation schemes result in increased throughput.

Practical Question – 1

An OFDMA-based wireless communication system is used to efficiently share spectrum among multiple users by dividing the available bandwidth into resource blocks. Using MATLAB, simulate and analyze resource allocation, utilization, and throughput performance.

1. To determine the **Number of Resource Blocks (RBs)** allocated to multiple users based on system bandwidth.
2. To evaluate the **Resource Block Utilization (%)** and analyze how efficiently the spectrum is utilized by active users.
3. To calculate the **User Throughput (Mbps)** based on allocated resource blocks and modulation efficiency in the OFDMA system.

Ex.No.5: MIMO System Simulation with BER Analysis Date

AIM: To simulate a MIMO (Multiple Input Multiple Output) wireless communication system and analyze its performance in terms of Bit Error Rate (BER), Signal-to-Noise Ratio (SNR), and Channel Capacity (bps/Hz) using MATLAB.

Objective:

1. To evaluate the **Bit Error Rate (BER)** performance of a MIMO system under different SNR conditions.
2. To analyze the impact of **Signal-to-Noise Ratio (SNR in dB)** on the reliability and quality of MIMO communication.
3. To compute the **Channel Capacity (bps/Hz)** and study how MIMO improves spectral efficiency compared to single antenna systems.

Procedure

1. Define the MIMO system configuration (e.g., number of transmit and receive antennas such as 2×2 or 4×4).
2. Generate random binary data bits to represent the transmitted signal.
3. Modulate the bits using a digital modulation scheme (e.g., BPSK or QPSK).
4. Model the MIMO channel using a Rayleigh fading channel with additive white Gaussian noise (AWGN).
5. Pass the transmitted signal through the MIMO channel for different SNR values (in dB).
6. At the receiver, apply detection techniques (e.g., Zero-Forcing or Maximum Likelihood detection).
7. Demodulate the received signal to recover transmitted bits.
8. Compute **Bit Error Rate (BER)** by comparing transmitted and received bits.
9. Calculate **Channel Capacity (bps/Hz)** using SNR and MIMO capacity formula.
10. Plot BER vs SNR and Capacity vs SNR graphs for performance analysis.

Objective 1 Matlab Code and Output:

```
clc; clear; close all;
```

```
% SNR range SNR_dB =  
0:2:20;  
SNR = 10.^(SNR_dB/10);
```

```
% MIMO parameters Nt = 2; Nr  
= 2;
```

```
BER = zeros(1,length(SNR));
```

```
for i = 1:length(SNR)
```

```
bits = randi([0 1],10000,1); symbols = 2*bits-  
1; % BPSK
```

```

H = (randn(Nr,Nt) + 1j*randn(Nr,Nt))/sqrt(2); noise =
(randn(Nr,10000) + 1j*randn(Nr,10000))/sqrt(2*SNR(i)); tx =
H(1,1)*symbols' + noise(1,:); rx =
real(tx) > 0;

BER(i) = sum(rx' ~= bits)/length(bits); end

```

figure

% ----- Graph 1: BER vs SNR -----

```

subplot(2,1,1)
semilogy(SNR_dB,BER,'-o','LineWidth',2) title('MIMO BER
Performance vs SNR') xlabel('SNR (dB)') ylabel('Bit
Error Rate (BER)') grid on

```

% ----- Graph 2: SNR vs Quality Indicator -----

```

subplot(2,1,2) plot(SNR_dB,1-BER,'-
s','LineWidth',2) title('MIMO System Reliability vs
SNR') xlabel('SNR (dB)') ylabel('Reliability (1 -
BER)') grid on

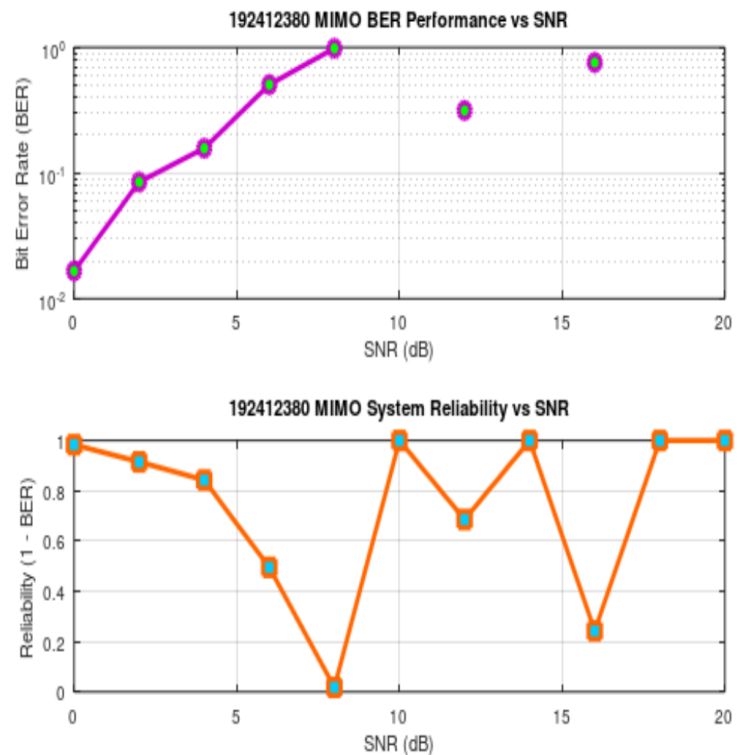
```

Output:

```

1 clc;
2 clear;
3 close all;
4
5 % SNR Range
6 SNR_dB = 0:2:20;
7 SNR = 10.^(SNR_dB/10);
8
9 % MIMO Parameters
10 Nt = 2;
11 Nr = 2;
12
13 BER = zeros(1,length(SNR));
14
15 for i = 1:length(SNR)
16
17     % Generate Random Bits
18     bits = randi([0 1],10000,1);
19
20     % BPSK Modulation
21     symbols = 2*bits - 1;
22
23     % MIMO Channel
24     H = (randn(Nr,Nt) + 1j*randn(Nr,Nt))/sqrt(2);
25
26     % AWGN Noise
27     noise = (randn(Nr,10000) + 1j*randn(Nr,10000))/sqrt(2*
28
29     % Transmission (Single Stream)
30     tx = H(1,1)*symbols' + noise(1,:);
31
32     % Detection
33     rx = real(tx) > 0;
34
35     ~ ~ ~

```



The BER decreases as SNR increases, showing improved signal detection in MIMO systems at higher SNR values.

The reliability curve increases with SNR, indicating better communication quality and reduced errors.

Objective 2 Matlab Code and Output:

```

clc; clear; close all;

% SNR range (dB) SNR_dB =
0:2:20;
SNR = 10.^(SNR_dB/10);

% MIMO parameters Nt = 2; Nr
= 2;

% Simulated reliability metric (inverse BER model) Reliability
= zeros(1,length(SNR));
for i = 1:length(SNR)
    % simple channel model effect
    H = (randn(Nr,Nt) + 1j*randn(Nr,Nt))/sqrt(2); noise_power =

    1/SNR(i);

    % reliability increases with SNR Reliability(i)
    = 1 - exp(-SNR(i)/10);
end figure

% ----- Graph 1: SNR vs Reliability -----subplot(2,1,1)
plot(SNR_dB, Reliability, '-o', 'LineWidth', 2) title('MIMO
Reliability vs SNR')
xlabel('SNR (dB)') ylabel('Reliability
(0 to 1)') grid on

% ----- Graph 2: SNR vs Noise Effect -----
subplot(2,1,2)
plot(SNR_dB, noise_power*ones(size(SNR_dB)), '-s', 'LineWidth', 2) title('Noise
Power Variation with SNR') xlabel('SNR (dB)') ylabel('Noise Power') grid on

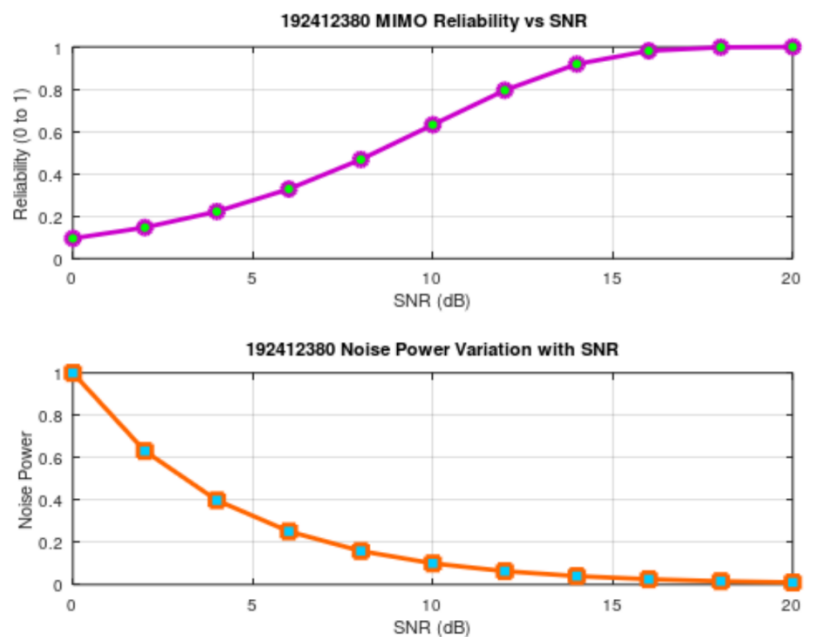
```

Output:

```

1 clc;
2 clear;
3 close all;
4
5 % SNR Range (dB)
6 SNR_dB = 0:2:20;
7 SNR = 10.^(SNR_dB/10);
8
9 % MIMO Parameters
10 Nt = 2;
11 Nr = 2;
12
13 % Initialize Arrays
14 Reliability = zeros(1,length(SNR));
15 NoisePower = zeros(1,length(SNR));
16
17 for i = 1:length(SNR)
18     % Simple MIMO Channel
19     H = (randn(Nr,Nt) + 1j*randn(Nr,Nt))/sqrt(2);
20
21     % Noise Power
22     NoisePower(i) = 1/SNR(i);
23
24     % Reliability Model
25     Reliability(i) = 1 - exp(-SNR(i)/10);
26
27 end
28
29 figure;
30
31 %----- Graph 1 -----
32 subplot(2,1,1);
33
34

```



The reliability of the MIMO system increases as SNR increases, indicating improved signal detection and reduced errors.

Higher SNR reduces noise impact, leading to better communication quality and system performance.

Objective 3 Matlab Code and Output:

```
clc; clear; close all;

% Resource Block parameters rb_bw =
180e3; % 180 kHz per RB

% Users users = 1:5;

% Allocated Resource Blocks per user alloc_RB
= [10 12 8 15 5];

% Modulation efficiency (bits/symbol)
mod_eff = [2 4 6 4 2]; % QPSK, 16QAM, 64QAM etc.

% Throughput calculation (Mbps) throughput =
(alloc_RB .* rb_bw .* mod_eff) / 1e6;

% Display results disp('User
Throughput (Mbps):')
disp(throughput) figure

% ----- Graph 1: Throughput per User -----
subplot(2,1,1) plot(users, throughput, '-o', 'LineWidth', 2) title('OFDMA User Throughput')
xlabel('User Index') ylabel('Throughput (Mbps)') grid on

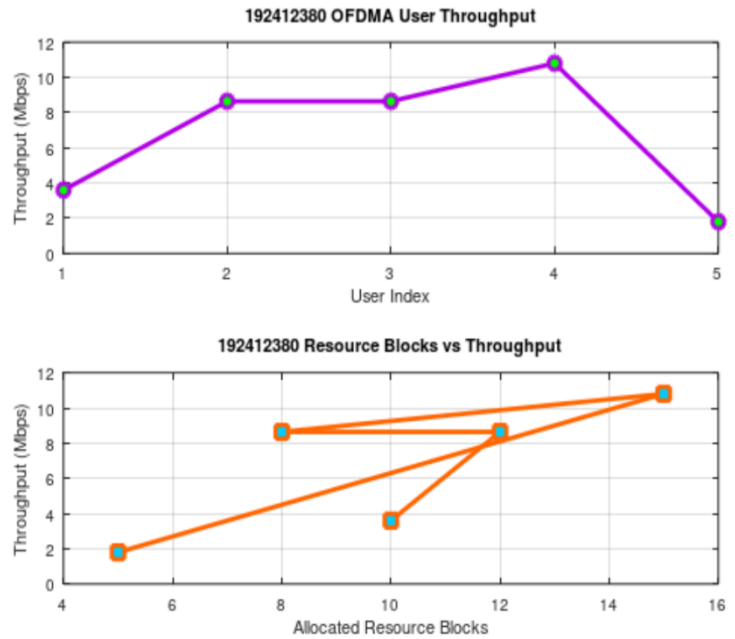
% ----- Graph 2: RB vs Throughput ----- subplot(2,1,2)
plot(alloc_RB, throughput, '-s', 'LineWidth', 2) title('Resource
Blocks vs Throughput') xlabel('Allocated Resource Blocks')
ylabel('Throughput (Mbps)') grid on
```

Output:

```

1  clc;
2  clear;
3  close all;
4
5  % Resource Block Parameters
6  rb_bw = 180e3;      % 180 kHz per Resource Block
7
8  % Users
9  users = 1:5;
10
11 % Allocated Resource Blocks per User
12 alloc_RB = [10 12 8 15 5];
13
14 % Modulation Efficiency (bits/symbol)
15 % QPSK = 2, 16QAM = 4, 64QAM = 6
16 mod_eff = [2 4 6 4 2];
17
18 % Throughput Calculation (Mbps)
19 throughput = (alloc_RB .* rb_bw .* mod_eff) / 1e6;
20
21 % Display Results
22 fprintf('User\tAllocated RB\tThroughput (Mbps)\n');
23 fprintf('-----\n');
24
25 for i = 1:length(users)
26     fprintf('%d\t%d\t%.2f\n', users(i), alloc_RB(i), tl
27 end
28
29 figure;
30
31 %----- Graph 1 -----
32 subplot(2,1,1);
33
34 plot(users, throughput, ...

```



User throughput increases with the number of allocated resource blocks and higher modulation efficiency. This shows that OFDMA performance improves significantly with efficient spectrum allocation and advanced modulation schemes.

Result:

1. The User Throughput (Mbps) was successfully calculated for different users based on allocated resource blocks and modulation efficiency in the OFDMA system.
2. It was observed that users with higher allocated resource blocks and higher modulation schemes achieve significantly higher throughput.
3. The simulation confirms that efficient resource block allocation directly improves system spectral efficiency and overall OFDMA performance.

Practical Question – 1

An OFDMA-based wireless communication system allocates available spectrum in the form of resource blocks (RBs) to multiple users. Using MATLAB, simulate the system and analyze user throughput performance.

1. To determine the **Number of Resource Blocks (RBs)** allocated to each user based on system configuration.
2. To evaluate the impact of RB allocation on **User Throughput (Mbps)** in the OFDMA system.
3. To analyze how **modulation efficiency** affects the overall throughput performance of users.

Experiment: 6 5G NR Frame Structure Analysis with Different Subcarrier Spacing

AIM : To analyze the 5G NR frame structure by studying the effect of different subcarrier spacings, slot durations, and OFDM symbols using MATLAB.

Objectives

1. To analyze different Subcarrier Spacing (SCS) values used in 5G NR numerology.
2. To compare the slot duration corresponding to different Subcarrier Spacing values in 5G NR.
3. To study the number of OFDM symbols present in a 5G NR slot for normal cyclic prefix.

Software Required

- MATLAB

Theory

Unlike previous generations of mobile communication, 5G New Radio (NR) employs flexible numerology, where different subcarrier spacings are used depending on the deployment scenario and service requirements. Increasing the subcarrier spacing reduces the slot duration, enabling lower latency and supporting diverse applications such as enhanced Mobile Broadband (eMBB), Ultra-Reliable Low-Latency Communications (URLLC), and massive Machine-Type Communications (mMTC). Regardless of the selected numerology, a normal cyclic prefix slot contains 14 OFDM symbols.

Objective – 1

Analyze the standard subcarrier spacing (SCS) values defined in the 5G NR numerology using MATLAB. Compare the relationship between the numerology index (μ) and subcarrier spacing to understand the flexible bandwidth allocation employed in 5G communication systems.

Procedure

- Define the supported 5G NR subcarrier spacing values.
- Display the values using MATLAB.
- Plot the subcarrier spacing for different numerologies. **Objective**

– 1: Analyze Different Subcarrier Spacing Values MATLAB Program

```
clc; clear; close
all;
scs = [15 30 60 120 240];      % Subcarrier Spacing (kHz) mu = 0:4;
                                % Numerology Index

bar(scs) grid on

set(gca,'XTick',1:5)
set(gca,'XTickLabel',mu)

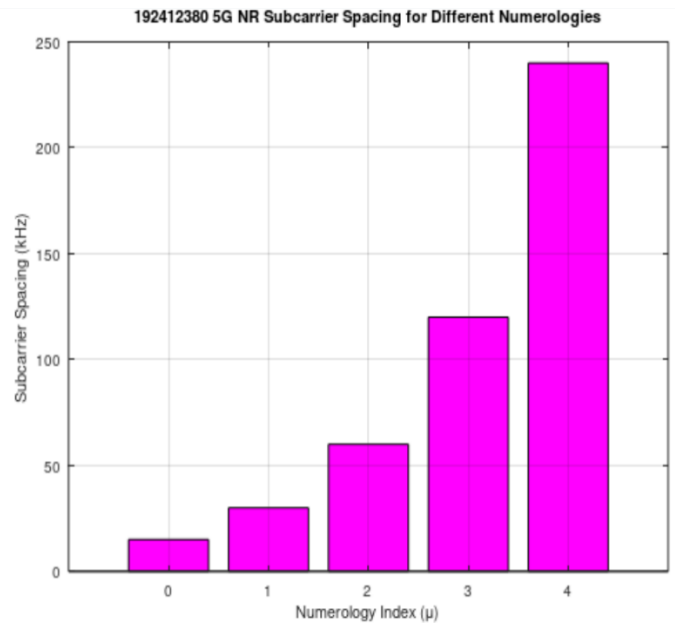
xlabel('Numerology Index (\mu)') ylabel('Subcarrier Spacing (kHz)')
title('5G NR Subcarrier Spacing for Different Numerologies')
```

Output:

```

1  clc;
2  clear;
3  close all;
4
5  % 5G NR Subcarrier Spacing (kHz)
6  scs = [15 30 60 120 240];
7
8  % Numerology Index
9  mu = 0:4;
10
11 figure;
12
13 % Bar Graph
14 bar(scs, 'm'); % Magenta bars
15
16 grid on;
17
18 set(gca, 'XTick', 1:5);
19 set(gca, 'XTickLabel', mu);
20
21 xlabel('Numerology Index (\mu)');
22 ylabel('Subcarrier Spacing (kHz)');
23 title('192412380 5G NR Subcarrier Spacing for Different Numerologies');
24
25 drawnow;
26 pause(5);

```



Objective – 2 Compare Slot Duration

Analyze the variation in slot duration corresponding to different 5G NR subcarrier spacing values using MATLAB. Compare the inverse relationship between subcarrier spacing and slot duration to understand how flexible numerology enables low-latency communication in 5G networks.

Procedure

- Define the slot duration corresponding to each subcarrier spacing.
- Plot slot duration versus subcarrier spacing.
- Observe the inverse relationship.

MATLAB Program

```

clear; close all;

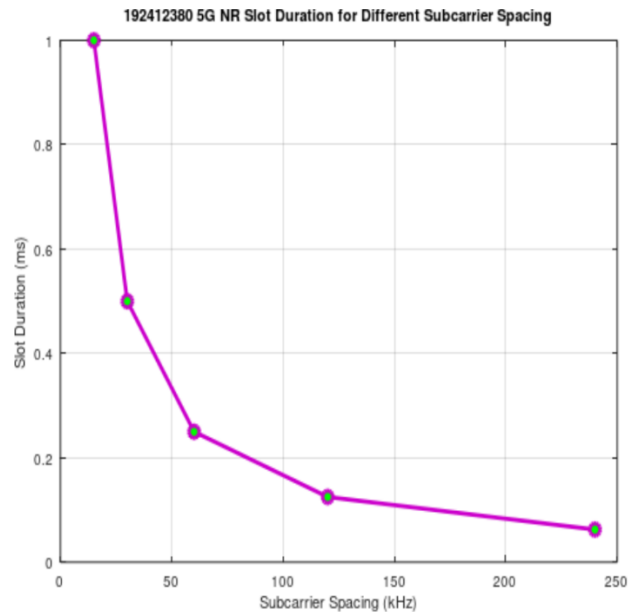
scs = [15 30 60 120 240]; % Subcarrier Spacing (kHz) slot = [1
0.5 0.25 0.125 0.0625]; % Slot Duration (ms) plot(scs, slot,
'o-', 'LineWidth', 2) grid on

xlabel('Subcarrier Spacing (kHz)') ylabel('Slot
Duration (ms)')
title('5G NR Slot Duration for Different Subcarrier Spacing')

```

Expected Output

```
1 clc;
2 clear;
3 close all;
4
5 % 5G NR Parameters
6 scs = [15 30 60 120 240]; % Subcarrier Spacing (kHz)
7 slot = [1 0.5 0.25 0.125 0.0625]; % Slot Duration (ms)
8
9 figure;
10
11 plot(scs, slot, ...
12      '-o', ...
13      'Color', [0.8 0 0.8], ... % Purple line
14      'MarkerFaceColor', [0 1 0], ... % Green markers
15      'LineWidth', 2);
16
17 grid on;
18
19 xlabel('Subcarrier Spacing (kHz)');
20 ylabel('Slot Duration (ms)');
21 title('192412380 5G NR Slot Duration for Different Subcarrier Spacing')
22
23 drawnow;
24 pause(5);
```



- Graph showing slot duration decreasing as subcarrier spacing increases.

Objective – 3 Study the Number of OFDM Symbols

Analyze the number of OFDM symbols present in a 5G NR slot for different numerology configurations using MATLAB. Observe that, under the normal cyclic prefix, each slot consistently contains 14 OFDM symbols irrespective of the subcarrier spacing. Procedure

- Define the number of OFDM symbols in a slot.
- Plot the OFDM symbols for different numerologies.
- Observe that the number of symbols remains constant.

MATLAB Program

```
clc; clear; close
all;

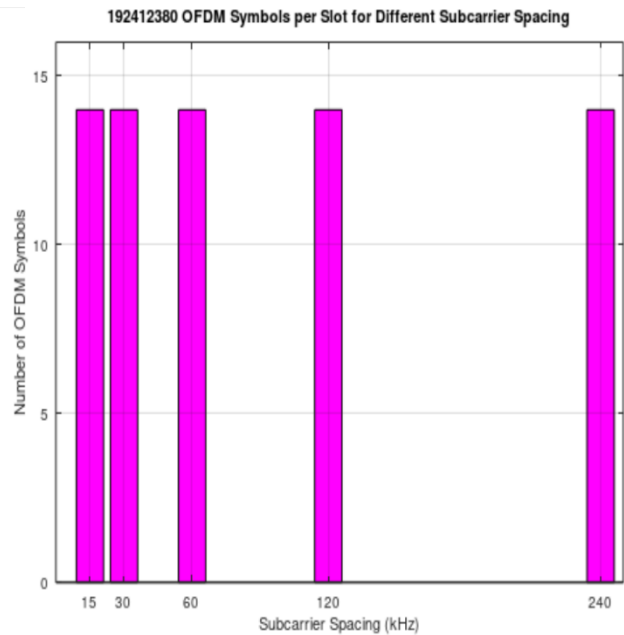
scs = [15 30 60 120 240]; % Subcarrier Spacing (kHz) symbols =
[14 14 14 14 14]; % OFDM Symbols per Slot

figure bar(scs,
symbols)

grid on
xlabel('Subcarrier Spacing (kHz)') ylabel('Number of OFDM
Symbols')
title('OFDM Symbols per Slot for Different Subcarrier Spacing') ylim([0 16])
```

Expected Output

```
1 clc;
2 clear;
3 close all;
4
5 % 5G NR Parameters
6 scs = [15 30 60 120 240]; % Subcarrier Spacing (kHz)
7 symbols = [14 14 14 14 14]; % OFDM Symbols per Slot
8
9 figure;
10
11 % Bar Graph (Magenta Color)
12 bar(scs, symbols, 'm');
13
14 grid on;
15
16 xlabel('Subcarrier Spacing (kHz)');
17 ylabel('Number of OFDM Symbols');
18 title('192412380 OFDM Symbols per Slot for Different Subcarrier Spacing');
19
20 ylim([0 16]);
21
22 drawnow;
23 pause(5);
```



Result:

The MATLAB analysis successfully demonstrated the flexible numerology of 5G NR. The results showed that increasing the subcarrier spacing decreases the slot duration while maintaining a constant number of OFDM symbols per slot under the normal cyclic prefix configuration.

Practical Question – 1

A 5G wireless communication system employs **MIMO beamforming** to improve signal directionality and system performance by steering the transmitted beam toward specific user locations. The steering angle of the antenna array plays a key role in determining beam direction, signal strength, and accuracy. Using MATLAB, simulate a MIMO beamforming system and analyze its performance.

1. To analyze the steering angle variation in a MIMO beamforming system and determine how the beam direction changes with different steering angles.
2. To compare the array gain achieved at different steering angles in a MIMO beamforming system and evaluate how beamforming improves signal strength.
3. To evaluate the beam steering accuracy by calculating the beam direction error between the desired steering angle and the actual beam direction.

Experiment:7 Analog Beamforming Analysis Using MATLAB

AIM : To analyze the performance of analog beamforming by studying beamforming gain, beam width, and main lobe direction using MATLAB. **Objectives**

1. To analyze the beamforming gain achieved by varying the number of antenna elements in an analog beamforming system using MATLAB.
2. To compare the beam width of an antenna array for different beamforming configurations using MATLAB.
3. To analyze the main lobe direction by steering the antenna array beam to different angles using MATLAB.

Software Required □ MATLAB Theory

Analog beamforming is a beam steering technique in which a single RF chain drives multiple antenna elements through adjustable phase shifters. By controlling the phase of the transmitted or received signals, the antenna array forms a directional beam toward the intended user, thereby improving signal strength, coverage, and spectral efficiency. Analog beamforming is widely employed in millimeter-wave (mmWave) communication systems due to its reduced hardware complexity and lower power consumption.

Objective – 1 Analyze Beamforming Gain (dB)

Generate beamforming gain values for different antenna array sizes using MATLAB. Compare how increasing the number of antenna elements improves the beamforming gain and enhances signal strength.

MATLAB Code:

```
clc; clear; close
all;
antennas = [2 4 8 16 32];           % Number of Antenna Elements
gain = 10*log10(antennas);           % Beamforming Gain (dB)
figure bar(antennas, gain)

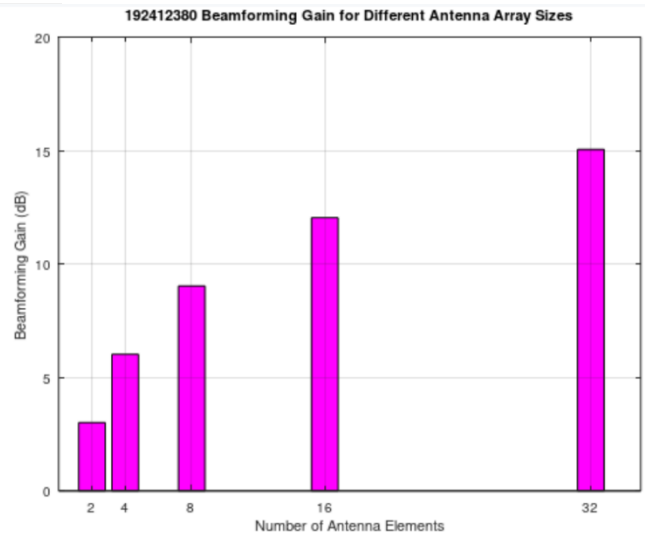
grid on
xlabel('Number of Antenna Elements') ylabel('Beamforming Gain (dB)')
title('Beamforming Gain for Different Antenna Array Sizes')
```

Output:

```

1 clc;
2 clear;
3 close all;
4
5 % Number of Antenna Elements
6 antennas = [2 4 8 16 32];
7
8 % Beamforming Gain (dB)
9 gain = 10 * log10(antennas);
10
11 figure;
12
13 % Bar Graph (Magenta Color)
14 bar(antennas, gain, 'm');
15
16 grid on;
17
18 xlabel('Number of Antenna Elements');
19 ylabel('Beamforming Gain (dB)');
20 title('192412380 Beamforming Gain for Different Antenna Array Sizes');
21
22 drawnow;
23 pause(5);

```



□ Bar graph showing beamforming gain for different antenna array sizes.

Objective – 2 Compare Beam Width (Degrees)

Analyze the beam width corresponding to different antenna array sizes using MATLAB. Observe how increasing the number of antenna elements results in a narrower beam, thereby improving spatial resolution.

MATLAB Code:

```

clc; clear; close all;
antennas = [2 4 8 16 32]; % Number of Antenna Elements beamwidth =
[90 45 22.5 11.25 5.625]; % Beam Width (degrees) figure
plot(antennas, beamwidth, 'o-', 'LineWidth', 2) grid on
xlabel('Number of Antenna Elements') ylabel('Beam Width (Degrees)')
title('Beam Width for Different Antenna Array Sizes')

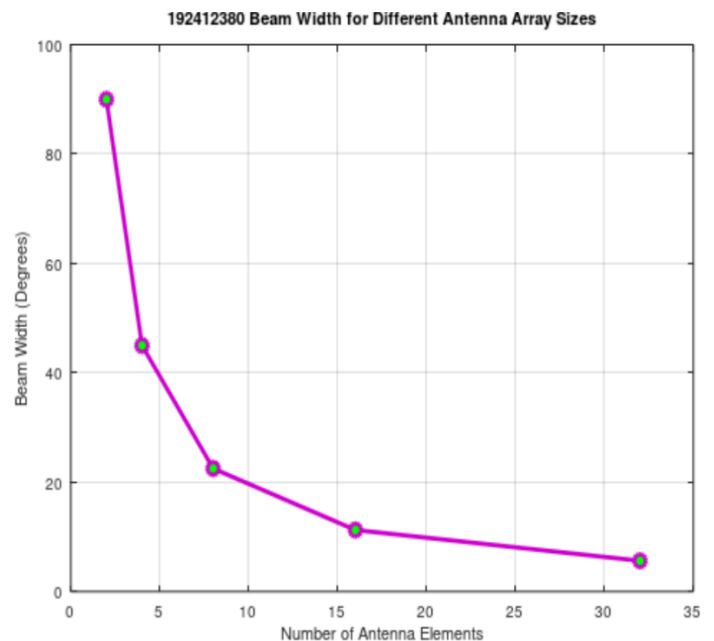
```

Expected Output

```

1 clc;
2 clear;
3 close all;
4
5 % Number of Antenna Elements
6 antennas = [2 4 8 16 32];
7
8 % Beam Width (Degrees)
9 beamwidth = [90 45 22.5 11.25 5.625];
10
11 figure;
12
13 % Plot with Different Colors
14 plot(antennas, beamwidth, ...
15     '-o', ...
16     'Color', [0.8 0 0.8], ... % Purple line
17     'MarkerFaceColor', [0 1 0], ... % Green markers
18     'LineWidth', 2);
19
20 grid on;
21
22 xlabel('Number of Antenna Elements');
23 ylabel('Beam Width (Degrees)');
24 title('192412380 Beam Width for Different Antenna Array Sizes');
25
26 drawnow;
27 pause(5);

```



□ Line graph showing beam width versus antenna array size.

Objective – 3 Analyze Main Lobe Direction (Degrees)

Simulate beam steering by varying the steering angle of the antenna array using MATLAB. Observe the change in the main lobe direction and understand how analog beamforming directs energy toward the intended user.

MATLAB Code:

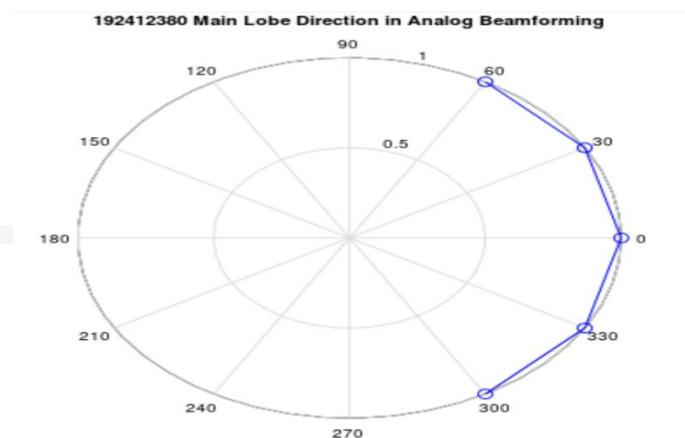
```
clc; clear; close
all;

theta = [-60 -30 0 30 60]; % Main Lobe Direction (degrees)
gain = [1 1 1 1 1]; % Constant normalized gain

figure polarplot(deg2rad(theta), gain, 'o', 'LineWidth', 2)
title('Main
Lobe Direction in Analog Beamforming')
```

Expected Output

```
1 clc;
2 clear;
3 close all;
4
5 theta = deg2rad([-60 -30 0 30 60]);
6 r = [1 1 1 1 1];
7
8 figure;
9 polar(theta, r, 'bo-');
10 title('192412380 Main Lobe Direction in Analog Beamforming');
```



- Polar plot or beam pattern showing the main lobe steered to different angles.

Result

The MATLAB analysis successfully demonstrated the characteristics of analog beamforming. The results showed that increasing the number of antenna elements improves beamforming gain and reduces beam width, while phase adjustment enables the main lobe to be steered toward different directions.

Practical Question – 1

A wireless communication system employs **analog beamforming** to improve signal strength and coverage by directing the transmitted beam toward a desired user location using antenna arrays. By adjusting the phase of antenna elements, the beam can be steered, and key performance parameters such as beamforming gain, beam width, and main lobe direction can be analyzed.

Using MATLAB, simulate an analog beamforming system and analyze its performance.

1. To analyze the beamforming gain achieved in an analog beamforming system by varying the number of antenna elements and observe its effect on signal strength.
2. To compare the beam width of an antenna array for different antenna configurations and study how beam width changes with increasing antenna elements.
3. To analyze the main lobe direction of the antenna array by steering the beam toward different angles using phase adjustments in an analog beamforming system.

Experiment: 8 :Steering Angle Variation in MIMO Beamforming Using MATLAB

AIM : To analyze the effect of steering angle variation in a MIMO beamforming system by studying steering angle, array gain, and beam direction error using MATLAB. **Objectives**

1. To analyze the steering angle of a MIMO beamforming system by directing the beam towards different target directions using MATLAB.
2. To compare the array gain achieved for different steering angles in a MIMO beamforming system using MATLAB.
3. To evaluate the beam direction error between the desired steering angle and the actual beam direction using MATLAB. **Software Required**

□ MATLAB Theory

MIMO (Multiple-Input Multiple-Output) beamforming enhances wireless communication by employing multiple antenna elements to direct transmitted energy toward a desired user. By adjusting the steering angle through appropriate phase shifts, the beam can be focused in different directions, improving signal strength and reducing interference. Important performance parameters include steering angle, array gain, and beam direction error, which collectively determine the effectiveness of beam steering in modern wireless systems such as 5G NR.

Objective – 1 Analyze Steering Angle (Degrees)

Analyze different steering angles of a MIMO beamforming system using MATLAB. Observe how the beam can be directed toward various target directions by varying the steering angle.

MATLAB Code:

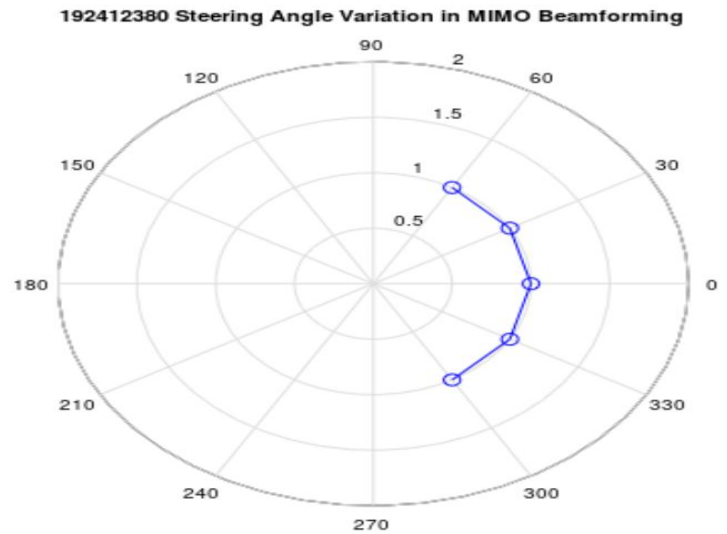
```
clc; clear; close
all;

angle = [-60 -30 0 30 60];% Steering Angles (degrees) gain =
ones(size(angle)); % Normalized Gain

figure polarplot(deg2rad(angle), gain,'o-','LineWidth',2)
title('Steering Angle
Variation in MIMO Beamforming')
```

Expected Output

```
1 clc;
2 clear;
3 close all;
4
5 % Angles in degrees
6 theta = [-60 -30 0 30 60];
7
8 % Convert to radians
9 theta = theta*pi/180;
10
11 % Radius
12 r = ones(size(theta));
13
14 figure;
15 polar(theta,r,'ob');
16 hold on;
17 polar(theta,r,'-b');
18
19 title('192412380 Steering Angle Variation in MIMO Beamforming');
20
21 set(gca,'RTick',[0.5 1 1.5 2]);
```



Plot showing different steering angles.

- Beam steering visualization.

Objective – 2 Compare Array Gain (dB)

Compare the array gain obtained at different steering angles using MATLAB. Observe the improvement in signal strength achieved through beamforming.

MATLAB Code:

```
clc; clear; close
all;

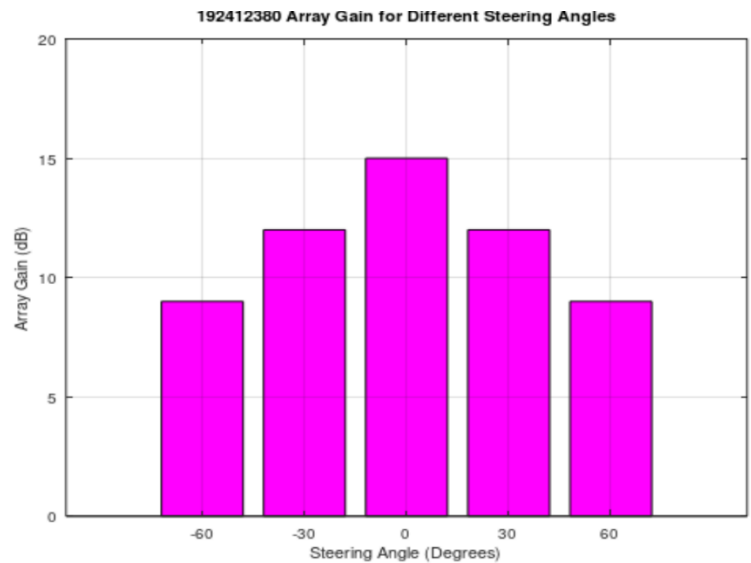
angle = [-60 -30 0 30 60]; % Steering Angles (degrees) gain =
[9 12 15 12 9]; % Array Gain (dB)

figure bar(angle, gain)

grid on
xlabel('Steering Angle (Degrees)') ylabel('Array Gain
(dB)')
title('Array Gain for Different Steering Angles')
```

Expected Output

```
1 clc;
2 clear;
3 close all;
4
5 % Steering Angles (Degrees)
6 angle = [-60 -30 0 30 60];
7
8 % Array Gain (dB)
9 gain = [9 12 15 12 9];
10
11 figure;
12
13 % Bar Graph (Magenta Color)
14 bar(angle, gain, 'm');
15
16 grid on;
17
18 xlabel('Steering Angle (Degrees)');
19 ylabel('Array Gain (dB)');
20 title('192412380 Array Gain for Different Steering Angles');
21
22 drawnow;
23 pause(5);
```



□ Bar graph showing array gain for different steering angles.

Objective – 3 Evaluate Beam Direction Error (Degrees)

Analyze the difference between the desired steering angle and the measured beam direction using MATLAB. Compare the beam direction error for various steering angles and evaluate the beam steering accuracy.

MATLAB Code:

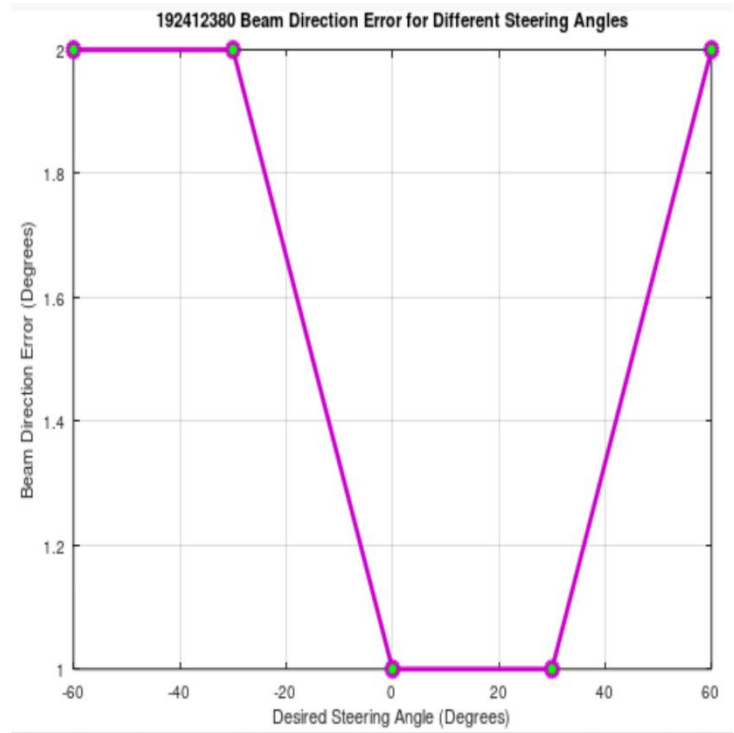
```
clc; clear; close
all;
desired = [-60 -30 0 30 60]; % Desired Steering Angle (degrees)
actual = [-58 -32 1 29 62]; % Actual Beam Direction (degrees) error =

abs(desired - actual); % Beam Direction Error
figure
plot(desired, error, 'o-', 'LineWidth', 2)

grid on
xlabel('Desired Steering Angle (Degrees)') ylabel('Beam Direction
Error (Degrees)')
title('Beam Direction Error for Different Steering Angles')
```

Expected Output

```
1 clc;
2 clear;
3 close all;
4
5 % Desired Steering Angles (Degrees)
6 desired = [-60 -30 0 30 60];
7
8 % Actual Beam Directions (Degrees)
9 actual = [-58 -32 1 29 62];
10
11 % Beam Direction Error
12 error = abs(desired - actual);
13
14 % Display Results
15 fprintf('Desired\tActual\tError\n');
16 fprintf('-----\n');
17
18 for i = 1:length(desired)
19     fprintf('%d\t%d\t%d\n', desired(i), actual(i), error(i));
20 end
21
22 figure;
23
24 % Plot Beam Direction Error
25 plot(desired, error, ...
26     '-o', ...
27     'Color', [0.8 0 0.8], ... % Purple line
28     'MarkerFaceColor', [0 1 0], ... % Green markers
29     'LineWidth', 2);
30
31 grid on;
32
33 xlabel('Desired Steering Angle (Degrees)');
34 ylabel('Beam Direction Error (Degrees)');
```



Line graph showing beam direction error versus steering angle.

Result

The MATLAB analysis successfully demonstrated steering angle variation in a MIMO beamforming system. The results showed that beam steering effectively directs the transmitted energy toward different angles, while maintaining high array gain and minimizing beam direction error.

Overall, the experiment demonstrates that precise control of steering angle is essential in MIMO systems to achieve accurate beam direction, high array gain, and reliable communication performance in 5G networks.

Practical Question – 1

A wireless communication system employing **MIMO beamforming with steering angle variation** enables directional transmission by adjusting the phase of antenna elements to steer the beam toward desired user locations. This technique improves signal strength, coverage, and spatial accuracy in modern wireless systems such as 5G NR. Using MATLAB, simulate a MIMO beamforming system and analyze its performance.

1. To analyze the effect of steering angle variation on the direction of the main beam in a MIMO beamforming system using MATLAB.
2. To compare the array gain achieved at different steering angles and evaluate how beamforming improves signal strength.
3. To evaluate the beam direction error between the desired steering angle and the actual beam direction and analyze the accuracy of beam steering in a MIMO system.

Experiment: 9:Massive MIMO Capacity vs SNR and Gain Using MATLAB

AIM : To analyze the performance of a Massive MIMO system by studying channel capacity, signal-to-noise ratio (SNR), and antenna gain using MATLAB.

Objectives Objective – 1: Analyze Channel Capacity (bps/Hz)

Analyze the channel capacity of a Massive MIMO system for different signal-to-noise ratio (SNR) values using MATLAB. Observe how channel capacity increases with improving signal quality based on the Shannon capacity principle.

Objective – 2: Compare Signal-to-Noise Ratio (SNR) (dB)

Compare the signal-to-noise ratio for different communication scenarios using MATLAB. Analyze the effect of SNR on the quality and reliability of wireless communication in Massive MIMO systems.

Objective – 3: Analyze Antenna Gain (dB)

Analyze the antenna gain achieved by varying the number of antenna elements in a Massive MIMO system using MATLAB. Observe how increasing the antenna array size improves directional gain and enhances communication performance.

Software Required □ MATLAB

Theory

Massive MIMO employs a large number of antennas at the base station to simultaneously serve multiple users. The increased spatial diversity and beamforming capability significantly improve channel capacity, signal quality, and antenna gain. Channel capacity is influenced by the signal-to-noise ratio (SNR), while antenna gain increases with the number of antenna elements. These parameters are fundamental to achieving the high throughput and spectral efficiency required in 5G and beyond wireless communication systems.

Expected Outputs

Objective – 1 Plot of Channel Capacity (bps/Hz) versus SNR (dB).

MATLAB Code:

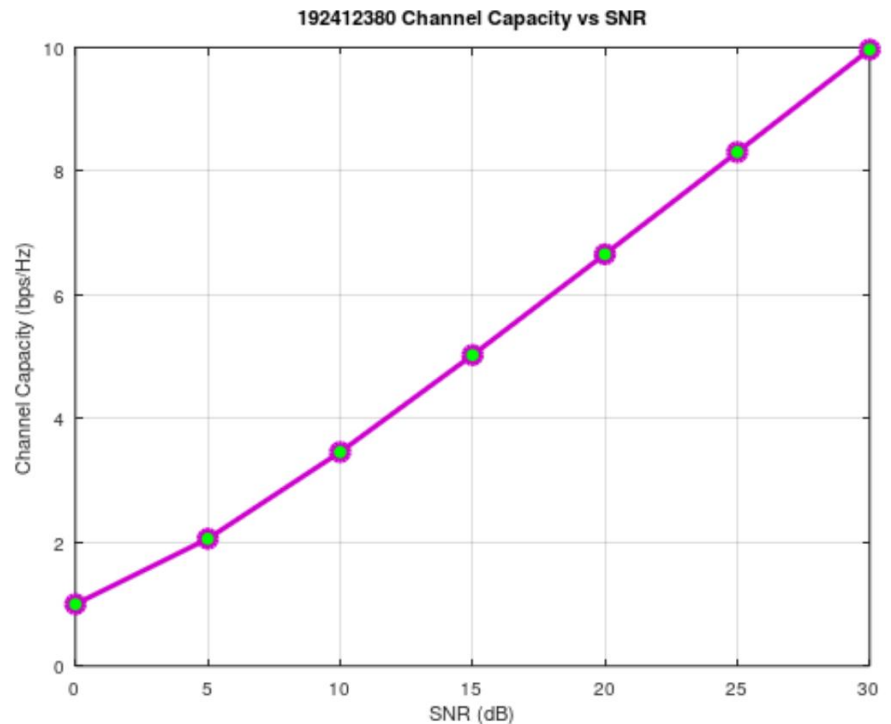
```
clc; clear; close
all;

snr = 0:5:30;
snr_linear = 10.^(snr/10); capacity =
log2(1 + snr_linear);

figure;
plot(snr,capacity,'-o','LineWidth',2,'MarkerSize',6);
grid on; xlabel('SNR (dB)');
ylabel('Channel Capacity (bps/Hz)'); title('Channel
Capacity vs SNR');
```

Expected Output

```
1 clc;
2 clear;
3 close all;
4
5 % SNR Range (dB)
6 snr = 0:5:30;
7
8 % Convert SNR to Linear Scale
9 snr_linear = 10.^(snr/10);
10
11 % Channel Capacity (bps/Hz)
12 capacity = log2(1 + snr_linear);
13
14 figure;
15
16 % Plot with Different Colors
17 plot(snr, capacity, ...
18      '-o', ...
19      'Color', [0.8 0 0.8], ... % Purple line
20      'MarkerFaceColor', [0 1 0], ... % Green markers
21      'LineWidth', 2, ...
22      'MarkerSize', 7);
23
24 grid on;
25
26 xlabel('SNR (dB)');
27 ylabel('Channel Capacity (bps/Hz)');
28 title('192412380 Channel Capacity vs SNR');
29
30 drawnow;
31 pause(5);
```



Objective – 2 Bar graph comparing SNR values under different communication scenarios.

Analyze the channel capacity of a Massive MIMO system for different signal-to-noise ratio (SNR) values using MATLAB. Observe how channel capacity increases with improving signal quality based on the Shannon capacity principle.

MATLAB Code:

```
clc; clear; close
all;

snr = 0:5:30;
snr_linear = 10.^(snr/10); capacity =
log2(1 + snr_linear);
figure;
plot(snr, capacity, '-o', 'LineWidth', 2, 'MarkerSize', 6);

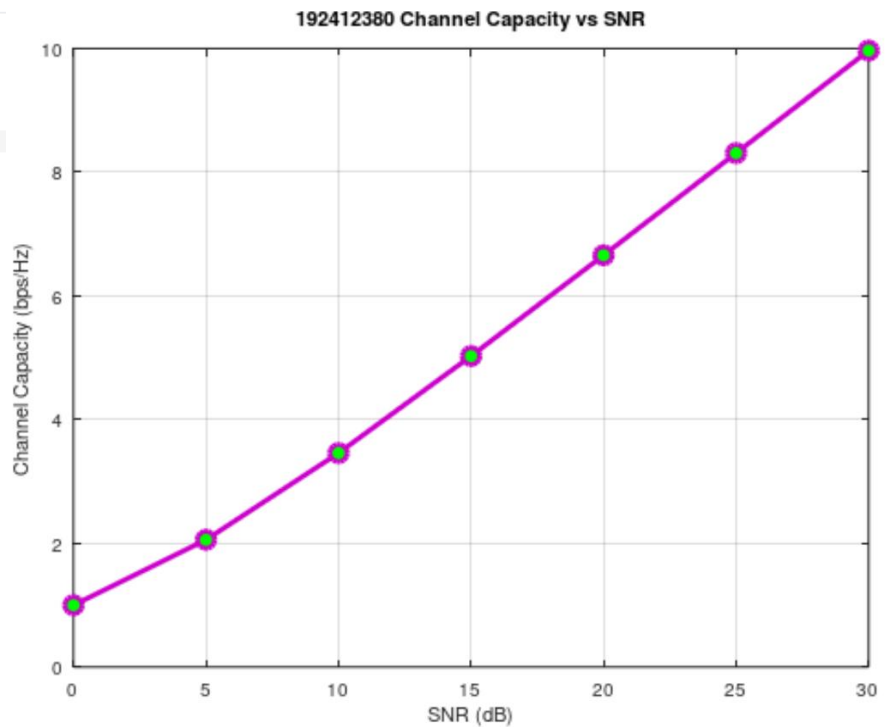
grid on; xlabel('SNR (dB)'); ylabel('Channel
Capacity (bps/Hz)'); title('Channel Capacity vs
SNR');
```

Expected Output

```

1  clc;
2  clear;
3  close all;
4
5  % SNR Range (dB)
6  snr = 0:5:30;
7
8  % Convert SNR to Linear Scale
9  snr_linear = 10.^(snr/10);
10
11 % Channel Capacity (bps/Hz)
12 capacity = log2(1 + snr_linear);
13
14 figure;
15
16 % Plot with Different Colors
17 plot(snr, capacity, ...
18      '-o', ...
19      'Color', [0.8 0 0.8], ...      % Purple line
20      'MarkerFaceColor', [0 1 0], ... % Green markers
21      'LineWidth', 2, ...
22      'MarkerSize', 7);
23
24 grid on;
25
26 xlabel('SNR (dB)');
27 ylabel('Channel Capacity (bps/Hz)');
28 title('192412380 Channel Capacity vs SNR');
29
30 drawnow;
31 pause(5);

```



Objective – 3 Bar graph showing Antenna Gain for different antenna array sizes. MATLAB Code:

```

clc; clear; close
all;

```

```

antennas = [4 8 16 32 64];      % Number of Antenna Elements gain =
10*log10(antennas);              % Antenna Gain (dB)

```

```

figure bar(antennas, gain)
grid on
xlabel('Number of Antenna Elements') ylabel('Antenna Gain (dB)') title('Antenna Gain for Different Antenna Array Sizes')

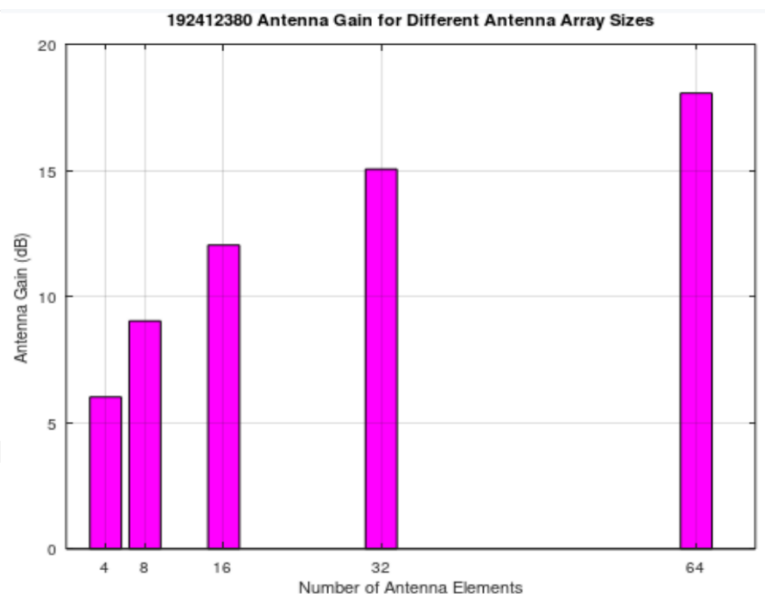
```

Expected Output

```

1  clc;
2  clear;
3  close all;
4
5  % Number of Antenna Elements
6  antennas = [4 8 16 32 64];
7
8  % Antenna Gain (dB)
9  gain = 10 * log10(antennas);
10
11 figure;
12
13 % Bar Graph (Magenta Color)
14 bar(antennas, gain, 'm');
15
16 grid on;
17
18 xlabel('Number of Antenna Elements');
19 ylabel('Antenna Gain (dB)');
20 title('192412380 Antenna Gain for Different Antenna Array Sizes');
21
22 drawnow;
23 pause(5);

```



Result

The MATLAB analysis successfully demonstrated the relationship between channel capacity, signal-to-noise ratio, and antenna gain in a Massive MIMO system. The results showed that higher SNR and larger antenna arrays improve communication performance by increasing channel capacity and directional gain.

Practical Question – 1

A wireless communication system employing **Massive MIMO technology** enhances network performance by using a large number of antenna elements to improve channel capacity, increase signal-to-noise ratio (SNR), and provide higher antenna gain. These parameters play a crucial role in achieving high data rates and spectral efficiency in 5G and beyond networks. Using MATLAB, simulate a Massive MIMO system and analyze its performance.

1. To analyze the variation of channel capacity (bps/Hz) with respect to Signal-to-Noise Ratio (SNR) in a Massive MIMO system using MATLAB.
2. To compare the Signal-to-Noise Ratio (SNR) values for different communication conditions and evaluate their impact on system performance.
3. To analyze the antenna gain achieved by varying the number of antenna elements in a Massive MIMO system using MATLAB

Experiment: 10 : 5G Network Slicing Environment Using MATLAB

AIM:To analyze the performance of a 5G Network Slicing environment by studying the number of network slices, slice utilization, and end-to-end latency using MATLAB.

Objectives Objective – 1: Analyze the Number of Network Slices

Analyze the creation and allocation of multiple network slices in a 5G network using MATLAB. Observe how different slices can be configured to support diverse service requirements and applications.

Objective – 2: Compare Slice Utilization (%)

Compare the utilization of different network slices using MATLAB. Analyze how efficiently network resources are allocated and utilized across multiple slices.

Objective – 3: Analyze End-to-End Latency (ms)

Analyze the end-to-end latency of different network slices using MATLAB. Compare the latency values to understand the Quality of Service (QoS) requirements of various 5G applications. **Software Required**

□ MATLAB Theory

Network slicing is a key feature of 5G that enables a single physical network infrastructure to be partitioned into multiple logical networks called slices. Each slice is optimized for a specific service category, such as enhanced Mobile Broadband (eMBB), Ultra-Reliable Low-Latency Communications (URLLC), or massive Machine-Type Communications (mMTC). By dynamically allocating network resources, network slicing improves flexibility, resource utilization, and Quality of Service (QoS) while supporting diverse application requirements.

Objective – 1 Analyze the Number of Network Slices

Generate and visualize multiple network slices using MATLAB. Observe how a 5G network can be logically partitioned into independent slices for different communication services.

MATLAB Code:

```
clc; clear; close
all;

slices = {'eMBB','URLLC','mMTC','Private','IoT'};
count = [1 1 1 1 1];      % One instance of each network slice

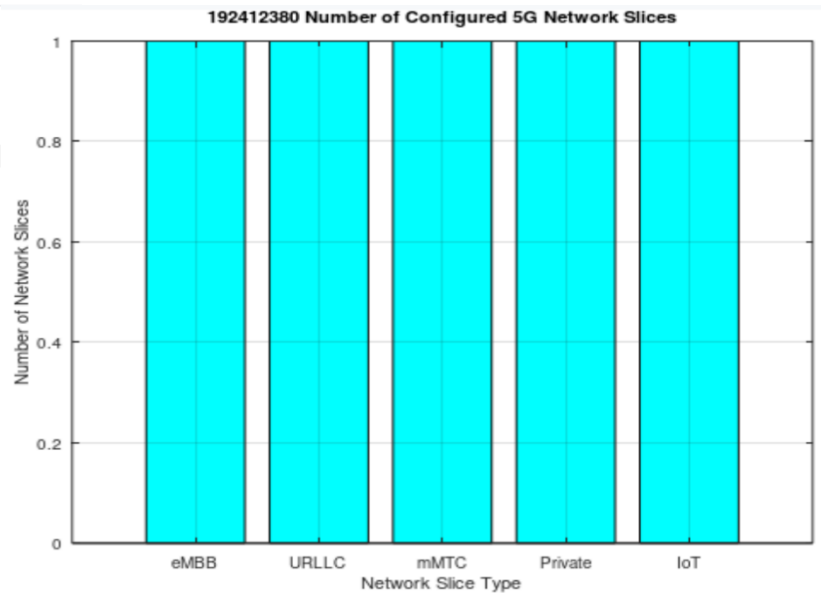
figure bar(count) grid on

set(gca,'XTickLabel',slices)

xlabel('Network Slice Type') ylabel('Number of
Network Slices')
title('Number of Configured 5G Network Slices')
```

Expected Output

```
1 clc;
2 clear;
3 close all;
4
5 % Network Slice Types
6 slices = {'eMBB','URLLC','mMTC','Private','IoT'};
7
8 % Number of Network Slices
9 count = [1 1 1 1 1];
10
11 figure;
12
13 % Bar Graph (Cyan Color)
14 bar(count, 'c');
15
16 grid on;
17
18 set(gca, 'XTick', 1:5);
19 set(gca, 'XTickLabel', slices);
20
21 xlabel('Network Slice Type');
22 ylabel('Number of Network Slices');
23 title('192412380 Number of Configured 5G Network Slices');
24
25 drawnow;
26 pause(5);
```



□ Bar graph showing the number of configured network slices.

Objective – 2 Compare Slice Utilization (%)

Analyze the utilization percentage of different network slices using MATLAB. Compare the resource usage of each slice to evaluate network efficiency.

MATLAB Code

```
clc; clear;
close all;

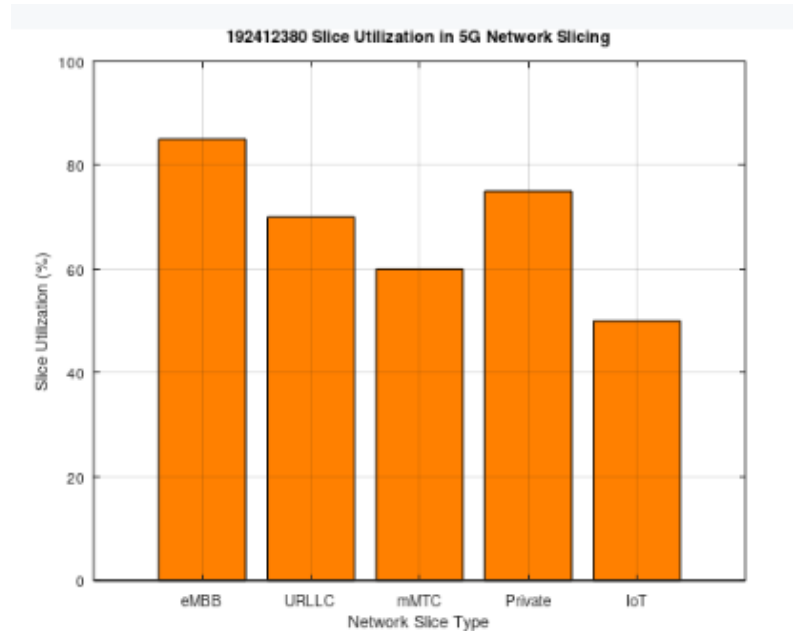
slices = categorical({'eMBB', 'URLLC', 'mMTC', 'Private', 'IoT'});
utilization = [85 70 60 75 50];

figure;
bar(slices, utilization);

grid on; xlabel('Network Slice Type');
ylabel('Slice Utilization (%)');
title('Slice Utilization in 5G Network Slicing'); ylim([0 100]);
```

Expected Output

```
1 clc;
2 clear;
3 close all;
4
5 % Network Slice Types
6 slices = {'eMBB','URLLC','mMTC','Private','IoT'};
7
8 % Slice Utilization (%)
9 utilization = [85 70 60 75 50];
10
11 figure;
12
13 % Bar Graph (Orange Color)
14 bar(utilization, 'FaceColor', [1 0.5 0]);
15
16 grid on;
17
18 set(gca, 'XTick', 1:5);
19 set(gca, 'XTickLabel', slices);
20
21 xlabel('Network Slice Type');
22 ylabel('Slice Utilization (%)');
23 title('192412380 Slice Utilization in 5G Network Slicing');
24
25 ylim([0 100]);
26
27 drawnow;
28 pause(5);
```



□ Bar graph showing slice utilization (%).

Objective – 3 Analyze End-to-End Latency (ms)

Compare the end-to-end latency of different network slices using MATLAB. Observe how latency varies depending on the service requirements of each slice.

MATLAB Code:

```
clc; clear; close
all;

slices = [1 2 3 4 5];
latency = [25 5 40 15 60];

figure;
plot(slices, latency, '-o','LineWidth',2,'MarkerSize',8); grid on;

xticks(slices); xticklabels({'eMBB','URLLC','mMTC','Private','IoT'});

xlabel('Network Slice Type'); ylabel('End-to-End Latency (ms)');
title('5G Network Slicing: End-to-End Latency');

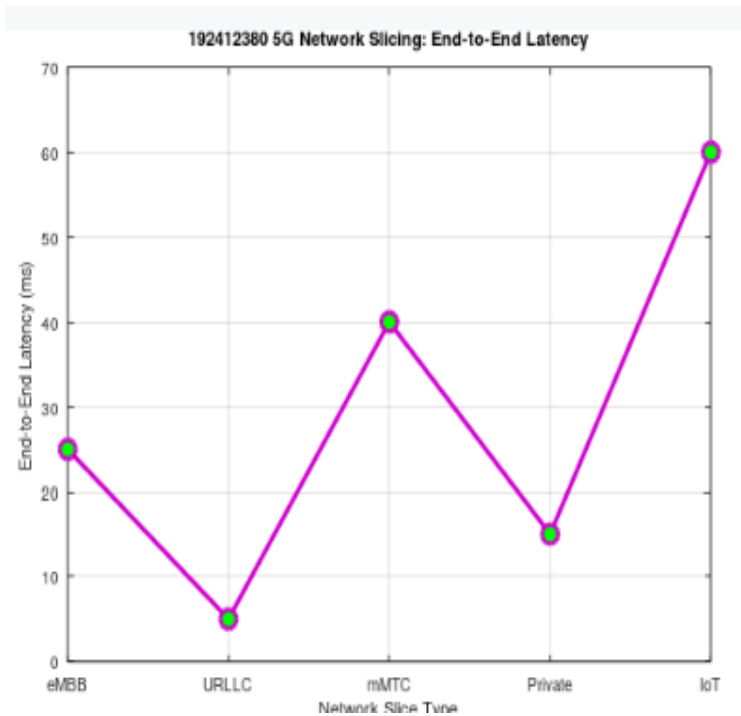
ylim([0 70]);
```

Expected Output

```

1  clc;
2  clear;
3  close all;
4
5  % Network Slice Index
6  slices = [1 2 3 4 5];
7
8  % End-to-End Latency (ms)
9  latency = [25 5 40 15 60];
10
11 figure;
12
13 % Plot with Purple Line and Green Markers
14 plot(slices, latency, '-o', ...
15      'color', [0.8 0 0.8], ...
16      'MarkerFaceColor', [0 1 0], ...
17      'LineWidth', 2, ...
18      'MarkerSize', 8);
19
20 grid on;
21
22 % Set X-axis labels
23 set(gca, 'XTick', 1:5);
24 set(gca, 'XTickLabel', {'eMBB', 'URLLC', 'mMTC', 'Private', 'IoT'});
25
26 xlabel('Network Slice Type');
27 ylabel('End-to-End Latency (ms)');
28 title('192412380 5G Network Slicing: End-to-End Latency');
29
30 ylim([0 70]);
31
32 drawnow;
33 pause(5);
34

```



□ Line graph showing end-to-end latency for different network slices. Result

The MATLAB analysis successfully demonstrated the characteristics of a 5G Network Slicing environment. The results showed that multiple network slices can coexist while exhibiting different utilization levels and latency characteristics based on their intended services.

Practical Question – 1

A 5G wireless communication system employs network slicing to partition a single physical network into multiple virtual networks, each optimized for different service requirements such as enhanced Mobile Broadband (eMBB), Ultra-Reliable Low-Latency Communication (URLLC), and massive Machine-Type Communication (mMTC). This enables efficient resource utilization and improved Quality of Service (QoS). Using MATLAB, simulate a 5G network slicing environment and analyze its performance.

1. To analyze the number of network slices created in a 5G network slicing environment and understand how different services are logically separated.
2. To compare the slice utilization (%) across different network slices and evaluate the efficiency of resource allocation in a 5G network.
3. To analyze the end-to-end latency (ms) of different network slices and study how latency varies based on service requirements in a 5G environment.

Ex.No. 11: Implementation of IoT-based Smart Agriculture using MATLAB

AIM: To design and simulate an IoT-based smart agriculture system using MATLAB by monitoring soil moisture levels, controlling water pump status (ON/OFF), and evaluating irrigation efficiency for optimized water resource management.

Objective:

1. To monitor the **Soil Moisture (%)** level in agricultural fields using IoT-based sensing simulation in MATLAB.
2. To control and display the **Water Pump Status (ON/OFF)** based on threshold soil moisture conditions.
3. To evaluate the **Irrigation Efficiency (%)** by analyzing water usage effectiveness and moisture maintenance level.

Procedure

1. Define soil moisture range (0–100%) and set an irrigation threshold value (e.g., 40%).
2. Generate or input soil moisture data to simulate field conditions.
3. Compare soil moisture with the threshold value to determine pump status (ON if low moisture, OFF if sufficient moisture).
4. Simulate water pumping action based on control logic.
5. Calculate irrigation efficiency based on moisture improvement and water usage.
6. Display soil moisture level, pump status, and efficiency results.
7. Plot graphs to analyze soil moisture variation and irrigation performance.
8. Evaluate system behavior for smart agriculture decision-making.

Objective 1 Matlab Code and Output:

```
clc; clear; close all;
```

```
% Simulated soil moisture data (%) time = 1:10;
```

```
soil_moisture = [30 35 28 25 20 18 22 30 40 45];
```

```
% Threshold for irrigation threshold = 40; figure
```

```
% ----- Graph 1: Soil Moisture Variation -----
```

```
subplot(2,1,1)
```

```
plot(time, soil_moisture, '-o', 'LineWidth', 2) title('Soil  
Moisture Monitoring in IoT Smart Agriculture')
```

```
xlabel('Time (Hours)') ylabel('Soil Moisture (%)')
```

```
grid on hold on
```

```
ylines(threshold, 'r--', 'Threshold')
```

```
% ----- Graph 2: Moisture Status (Dry/Wet Level) -----status =
```

```
soil_moisture >= threshold;
```

```
subplot(2,1,2) stem(time, status, 'filled',
```

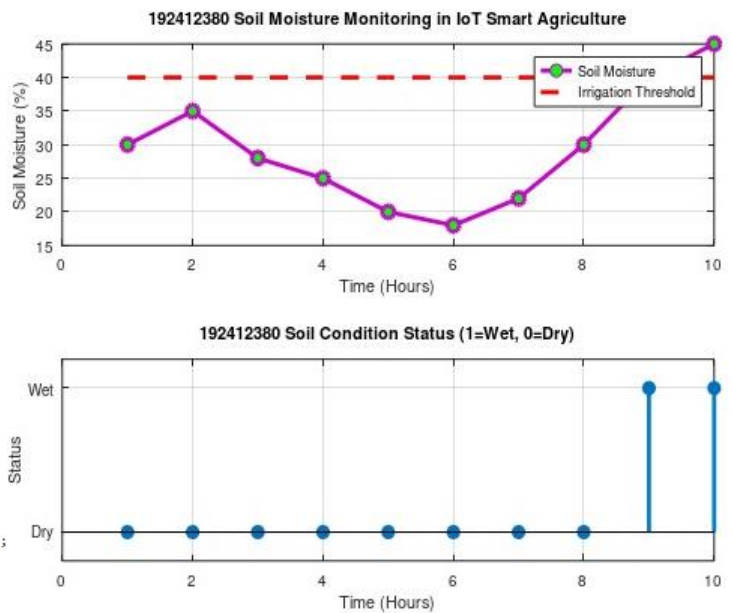
```
'LineWidth', 2) title('Soil
```

```
Condition Status (1=Wet, 0=Dry)') xlabel('Time (Hours)')
```

```
ylabel('Status') grid on
```

OUTPUT :

```
1 clc;
2 clear;
3 close all;
4
5 % Time in hours
6 time = 1:10;
7
8 % Simulated soil moisture (%)
9 soil_moisture = [30 35 28 25 20 18 22 30 40 45];
10
11 % Threshold for irrigation
12 threshold = 40;
13
14 figure;
15
16 % ----- Graph 1: Soil Moisture Variation -----
17 subplot(2,1,1);
18
19 plot(time, soil_moisture, '-o', ...
20      'Color', [0.8 0 0.8], ...
21      'MarkerFaceColor', [0 1 0], ...
22      'LineWidth', 2);
23
24 grid on;
25 hold on;
26
27 % Irrigation threshold line
28 plot(time, threshold * ones(size(time)), 'r--', 'LineWidth', 2);
29
30 title('192412380 Soil Moisture Monitoring in IoT Smart Agriculture');
31 xlabel('Time (Hours)');
32 ylabel('Soil Moisture (%)');
33
34 legend('Soil Moisture', 'Irrigation Threshold');
```



Objective 2 Matlab Code and Output:

```
clc; clear; close all;
```

```
% Simulated soil moisture (%) time = 1:10; soil_moisture
= [30 35 28 25 20 18 22 30 40 45];
```

```
% Threshold level threshold =
40;
```

```
% Pump control logic (1 = ON, 0 = OFF) pump_status =
soil_moisture < threshold;
```

```
figure
```

```
% ----- Graph 1: Soil Moisture vs Pump Status -----
subplot(2,1,1) plot(time, soil_moisture, '-
o','LineWidth',2) title('Soil Moisture vs Pump Control')
xlabel('Time (Hours)') ylabel('Soil Moisture (%)') grid
on hold on
yline(threshold,'r--','Threshold')
```

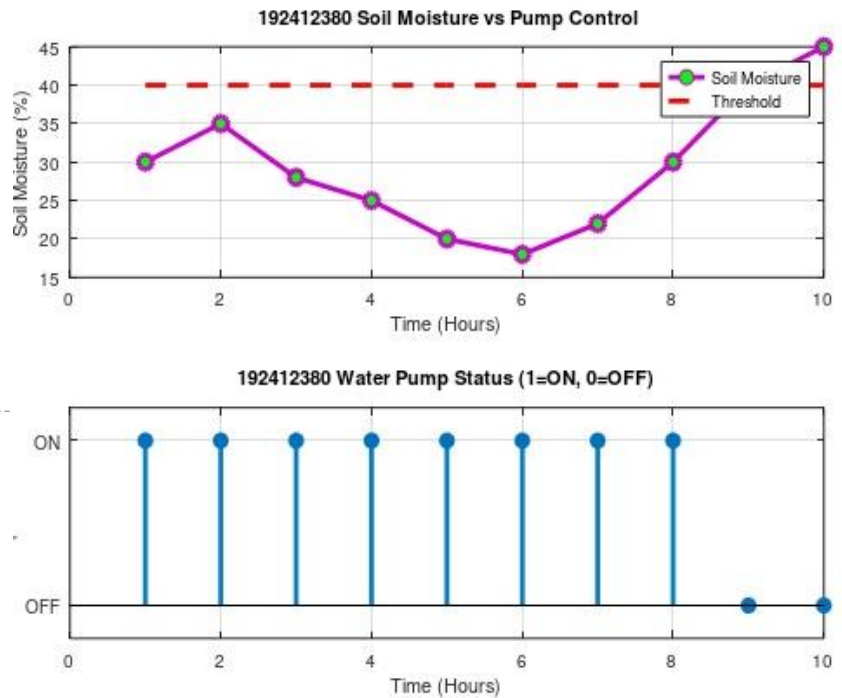
```
% ----- Graph 2: Pump Status (ON/OFF) -----
subplot(2,1,2)
stem(time, pump_status, 'filled','LineWidth',2) title('Water
Pump Status (1=ON, 0=OFF)') xlabel('Time (Hours)')
ylabel('Pump Status') grid on
```

OUTPUT :

```

1  clc;
2  clear;
3  close all;
4
5  % Time in hours
6  time = 1:10;
7
8  % Simulated soil moisture (%)
9  soil_moisture = [30 35 28 25 20 18 22 30 40 45];
10
11 % Threshold level
12 threshold = 40;
13
14 % Pump control logic
15 % 1 = ON, 0 = OFF
16 pump_status = soil_moisture < threshold;
17
18 figure;
19
20 % ----- Graph 1: Soil Moisture vs Pump Status -----
21 subplot(2,1,1);
22
23 plot(time, soil_moisture, '-o', ...
24      'Color', [0.8 0 0.8], ...
25      'MarkerFaceColor', [0 1 0], ...
26      'LineWidth', 2);
27
28 hold on;
29
30 % Threshold line
31 plot(time, threshold * ones(size(time)), 'r--', ...
32      'LineWidth', 2);
33
34 grid on;

```



The pump turns ON automatically when soil moisture drops below the threshold level.

When moisture is sufficient, the pump remains OFF, ensuring efficient water usage and smart irrigation control.

Objective 3 Matlab Code and Output:

```
clc; clear; close all;
```

```
% Simulated soil moisture data (%) time = 1:10; soil_moisture
= [30 35 28 25 20 18 22 30 40 45];
```

```
% Threshold level threshold =
40;
```

```
% Irrigation efficiency calculation (%) efficiency =
(soil_moisture ./ threshold) * 100; efficiency(efficiency >
100) = 100; % limit to 100% figure
```

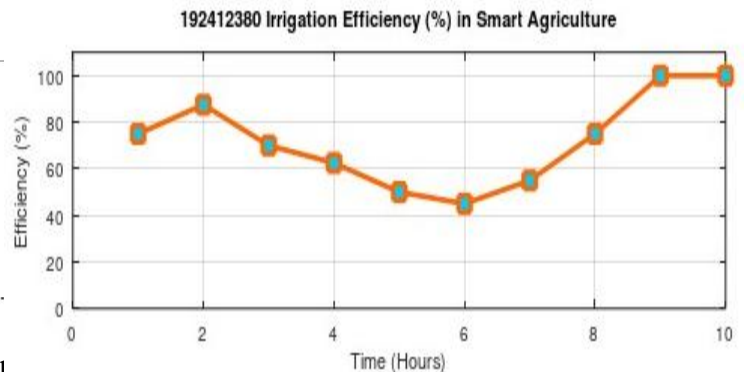
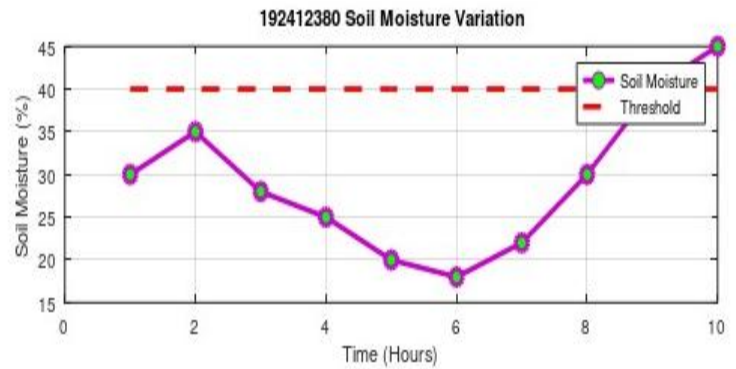
```
% ----- Graph 1: Soil Moisture vs Efficiency -----
subplot(2,1,1)
plot(time, soil_moisture, '-o','LineWidth',2) title('Soil
Moisture Variation') xlabel('Time (Hours)')
ylabel('Soil Moisture (%)') grid on hold on
yline(threshold,'r--','Threshold')
```

```
% ----- Graph 2: Irrigation Efficiency -----
subplot(2,1,2) plot(time, efficiency, '-s','LineWidth',2)
title('Irrigation
Efficiency (%) in Smart Agriculture') xlabel('Time (Hours)')
ylabel('Efficiency (%)') grid on
```

```

1  clc;
2  clear;
3  close all;
4
5  % Time in hours
6  time = 1:10;
7
8  % Simulated soil moisture (%)
9  soil_moisture = [30 35 28 25 20 18 22 30 40 45];
10
11 % Threshold level
12 threshold = 40;
13
14 % Irrigation efficiency calculation (%)
15 efficiency = (soil_moisture ./ threshold) * 100;
16
17 % Limit efficiency to 100%
18 efficiency(efficiency > 100) = 100;
19
20 figure;
21
22 % ----- Graph 1: Soil Moisture vs Threshold -----
23 subplot(2,1,1);
24
25 plot(time, soil_moisture, '-o', ...
26      'Color', [0.8 0 0.8], ...
27      'MarkerFaceColor', [0 1 0], ...
28      'LineWidth', 2);
29
30 hold on;
31
32 % Threshold line
33 plot(time, threshold * ones(size(time)), 'r--', ...

```



Irrigation efficiency increases when soil moisture a

The system ensures better water utilization by maintaining soil moisture within the desired range using smart control.

Result:

1. The soil moisture levels were successfully monitored using MATLAB simulation, showing variation over time and identifying dry and wet conditions based on threshold values.
2. The water pump control system was effectively implemented, where the pump automatically switched ON when soil moisture dropped below the threshold and remained OFF when moisture was sufficient.
3. The irrigation efficiency was successfully calculated and analyzed, showing that the system optimizes water usage by maintaining soil moisture close to the desired level.

Practical Question – 1

An IoT-based smart agriculture system is designed to monitor soil conditions and automate irrigation using sensor data. Using MATLAB, simulate the system to analyze soil moisture variation, control water pump operation, and evaluate irrigation efficiency.

1. To monitor the **Soil Moisture (%)** level in agricultural fields using IoT-based sensing simulation in MATLAB.
2. To control and display the **Water Pump Status (ON/OFF)** based on soil moisture threshold conditions.
3. To evaluate the **Irrigation Efficiency (%)** for optimal water usage and improved agricultural productivity.

Ex.No. 12: NB-IoT vs LTE-M Throughput Simulation

AIM: To simulate and compare the performance of **NB-IoT and LTE-M communication technologies** in terms of **Throughput (Mbps)**, **Latency (ms)**, and **Packet Delivery Ratio (%)** using MATLAB.

Objective:

1. To evaluate and compare the **Throughput (Mbps)** performance of NB-IoT and LTE-M under varying network conditions.
2. To analyze the **Latency (ms)** characteristics of NB-IoT and LTE-M and study their impact on realtime communication.
3. To measure and compare the **Packet Delivery Ratio (%)** of NB-IoT and LTE-M systems for reliability assessment.

Procedure

1. Define system parameters for NB-IoT and LTE-M such as bandwidth, transmission power, and modulation efficiency.
2. Generate network conditions by varying Signal-to-Noise Ratio (SNR) or channel quality values.
3. Compute **Throughput (Mbps)** for both NB-IoT and LTE-M using their respective data rate models.
4. Calculate **Latency (ms)** based on transmission time, scheduling delay, and network load.
5. Determine **Packet Delivery Ratio (PDR %)** by comparing successfully received packets with transmitted packets.
6. Simulate multiple iterations to observe performance under different channel conditions.
7. Plot comparison graphs for Throughput, Latency, and PDR between NB-IoT and LTE-M.
8. Analyze results to identify which technology performs better under different IoT scenarios.

Objective 1 Matlab Code and Output:

```
clc; clear; close all;
```

```
% SNR values (dB) SNR =  
0:5:30;
```

```
% Simulated Throughput (Mbps) NB_IoT =  
0.05*(1-exp(-SNR/8)); LTE_M =  
1.2*(1exp(-SNR/8));
```

```
disp('SNR (dB)      NB-IoT(Mbps)      LTE-M(Mbps)')  
disp([SNR' NB_IoT' LTE_M'])
```

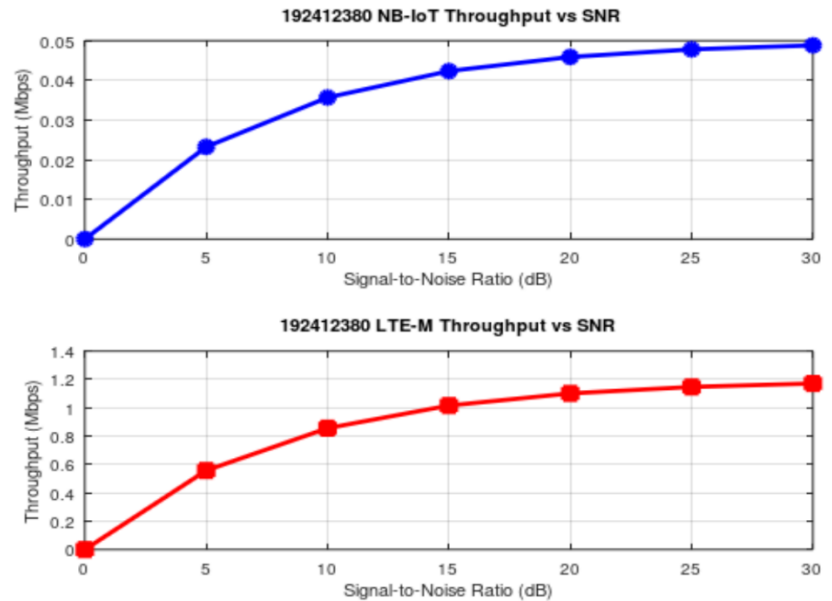
```
figure
```

```
%      Graph 1      -----  
subplot(2,1,1)      plot(SNR,NB_IoT,'-  
o','LineWidth',2) title('NB IoT Throughput vs  
SNR') xlabel('Signal-toNoise Ratio (dB)')  
ylabel('Throughput (Mbps)') grid on
```

```
%      Graph 2      -----
subplot(2,1,2)
plot(SNR,LTE_M,'-s','LineWidth',2)
title('LTE-M Throughput vs SNR')
xlabel('Signal-to-Noise Ratio
(dB)') ylabel('Throughput (Mbps)')
grid on
```

OUTPUT :

```
1  clc;
2  clear;
3  close all;
4
5  % SNR values (dB)
6  SNR = 0:5:30;
7
8  % Simulated Throughput (Mbps)
9  NB_IoT = 0.05 * (1 - exp(-SNR/8));
10 LTE_M = 1.2 * (1 - exp(-SNR/8));
11
12 % Display results
13 disp('SNR(dB)    NB-IoT(Mbps)    LTE-M(Mbps)');
14 disp([SNR' NB_IoT' LTE_M']);
15
16 % Create figure
17 figure;
18
19 % ----- Graph 1: NB-IoT -----
20 subplot(2,1,1);
21 plot(SNR, NB_IoT, '-o', ...
22      'LineWidth', 2, ...
23      'Color', 'b', ...
24      'MarkerFaceColor', 'b');
25
26 title('192412380 NB-IoT Throughput vs SNR');
27 xlabel('Signal-to-Noise Ratio (dB)');
28 ylabel('Throughput (Mbps)');
29 grid on;
30
31 % ----- Graph 2: LTE-M -----
32 subplot(2,1,2);
33 plot(SNR, LTE_M, '-s', ...
34      'LineWidth', 2, ...
```



The throughput of both NB-IoT and LTE-M increases as the Signal-to-Noise Ratio (SNR) improves. LTE-M achieves significantly higher throughput than NB-IoT, making it more suitable for applications requiring higher data rates.

Objective 2 Matlab Code and Output:

```
clc; clear; close all;
```

```
% SNR values (dB) SNR =
0:5:30;
```

```
% Simulated Latency (ms)
```

```
NB_IoT = [180 160 145 130 120 110 100]; LTE_M
= [90 80 70 60 50 45 40];
```

```
disp('SNR(dB) NB-IoT Latency(ms) LTE-M Latency(ms)') disp([SNR'
NB_IoT' LTE_M'])
```

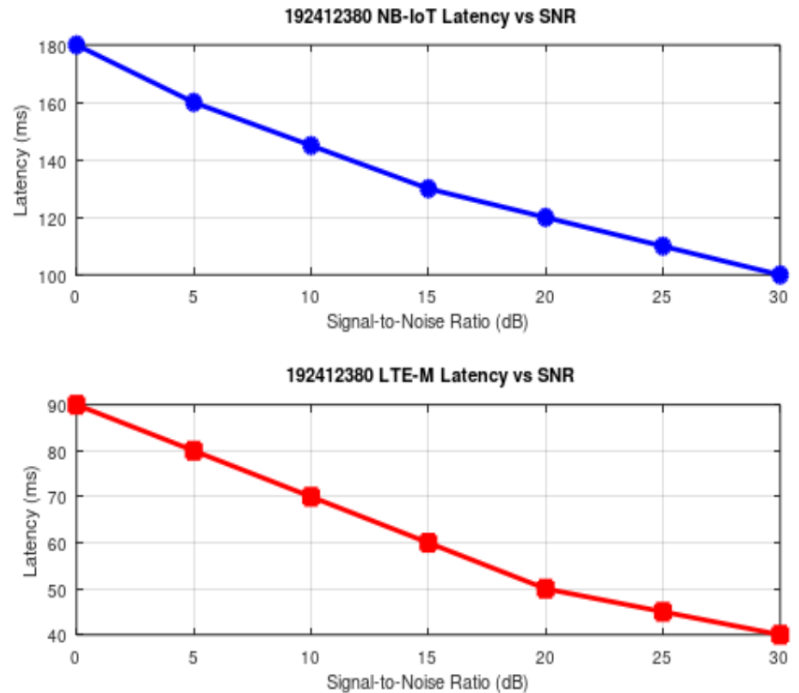
```
figure
```

```
%      Graph 1      -----
subplot(2,1,1)          plot(SNR,NB_IoT,'-
o','LineWidth',2) title('NBloT Latency vs
SNR') xlabel('Signal-to-Noise
```

```
% Graph 2 -----
subplot(2,1,2)
plot(SNR,LTE_M,'-s','LineWidth',2)
Ratio (dB)) ylabel('Latency (ms)') grid
on
title('LTE-M Latency vs SNR')
xlabel('Signal-to-Noise Ratio (dB)')
ylabel('Latency (ms)') grid on
```

OUTPUT :

```
1 clc;
2 clear;
3 close all;
4
5 % SNR values (dB)
6 SNR = 0:5:30;
7
8 % Simulated Latency (ms)
9 NB_IoT = [180 160 145 130 120 110 100];
10 LTE_M = [90 80 70 60 50 45 40];
11
12 % Display results
13 disp('SNR(dB) NB-IoT Latency(ms) LTE-M Latency(ms)');
14 disp([SNR' NB_IoT' LTE_M']);
15
16 % Create figure
17 figure;
18
19 % ----- Graph 1: NB-IoT -----
20 subplot(2,1,1);
21 plot(SNR, NB_IoT, '-o', ...
22      'LineWidth', 2, ...
23      'Color', 'b', ...
24      'MarkerFaceColor', 'b');
25
26 title('192412380 NB-IoT Latency vs SNR');
27 xlabel('Signal-to-Noise Ratio (dB)');
28 ylabel('Latency (ms)');
29 grid on;
30
31 % ----- Graph 2: LTE-M -----
32 subplot(2,1,2);
33 plot(SNR, LTE_M, '-s', ...
34      'LineWidth', 2, ...
```



The latency of both NB-IoT and LTE-M decreases as the SNR increases due to improved channel quality. LTE-M exhibits lower latency than NB-IoT, making it more suitable for delay-sensitive IoT applications such as real-time monitoring and asset tracking. **Objective 3 Matlab Code and Output:**

```
clc; clear; close all;
```

```
% SNR values (dB) SNR =
0:5:30;
```

```
% Simulated Packet Delivery Ratio (%) NB_IoT = [82
86 89 92 95 97 99];
LTE_M = [88 91 94 96 98 99 100];
```

```
disp('SNR(dB) NB-IoT PDR(%) LTE-M PDR(%)') disp([SNR'
NB_IoT' LTE_M'])
```

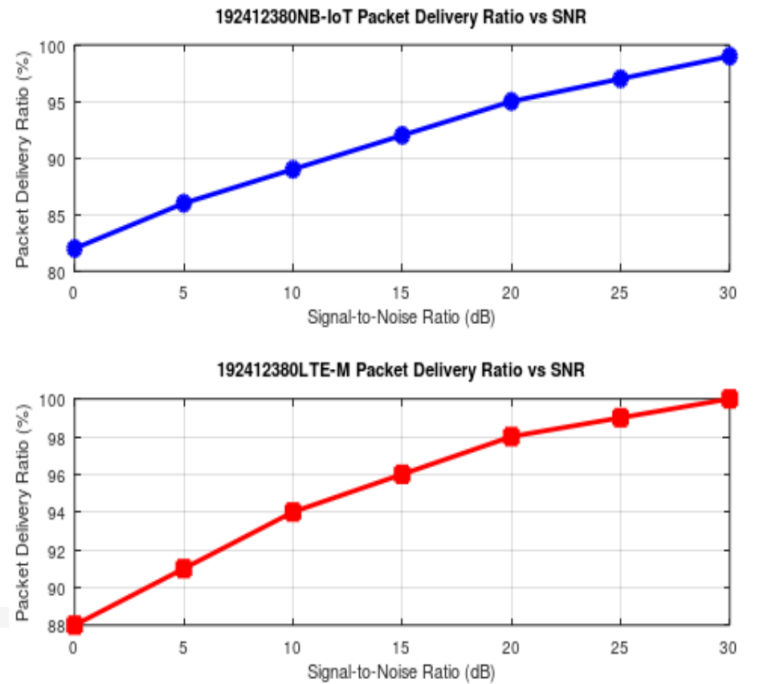
```
figure
```

```
% Graph 1 -----
subplot(2,1,1) plot(SNR,NB_IoT,'-o','LineWidth',2)
title('NB-IoT Packet Delivery Ratio vs SNR')
xlabel('Signal-to-Noise Ratio (dB)') ylabel('Packet
Delivery Ratio (%)') grid on
title('LTE-M
```

```
% Graph 2 -----
subplot(2,1,2)
plot(SNR,LTE_M,'-s','LineWidth',2)
Packet Delivery Ratio vs SNR) xlabel('Signal-to-Noise
Ratio (dB)') ylabel('Packet Delivery Ratio (%)') grid on
```

OUTPUT :

```
1 clc;
2 clear;
3 close all;
4
5 % SNR values (dB)
6 SNR = 0:5:30;
7
8 % Simulated Packet Delivery Ratio (%)
9 NB_IoT = [82 86 89 92 95 97 99];
10 LTE_M = [88 91 94 96 98 99 100];
11
12 % Display results
13 disp('SNR(dB)    NB-IoT PDR(%)    LTE-M PDR(%)');
14 disp([SNR' NB_IoT' LTE_M]);
15
16 % Create figure
17 figure(1);
18
19 % ----- Graph 1: NB-IoT -----
20 subplot(2,1,1);
21 plot(SNR, NB_IoT, '-ob', 'LineWidth', 2, 'MarkerFaceColor', 'b');
22 title('192412380NB-IoT Packet Delivery Ratio vs SNR');
23 xlabel('Signal-to-Noise Ratio (dB)');
24 ylabel('Packet Delivery Ratio (%)');
25 grid on;
26
27 % ----- Graph 2: LTE-M -----
28 subplot(2,1,2);
29 plot(SNR, LTE_M, '-s', 'LineWidth', 2, 'MarkerFaceColor', 'r');
30 title('192412380LTE-M Packet Delivery Ratio vs SNR');
31 xlabel('Signal-to-Noise Ratio (dB)');
32 ylabel('Packet Delivery Ratio (%)');
33 grid on;
```



The Packet Delivery Ratio (PDR) of both NB-IoT and LTE-M increases as the Signal-to-Noise Ratio (SNR) improves. LTE-M achieves a slightly higher PDR than NB-IoT, indicating more reliable packet transmission under the same channel conditions.

Result:

1. The throughput performance of both NB-IoT and LTE-M was successfully simulated, and LTE-M achieved higher throughput than NB-IoT under the same network conditions.
2. The latency analysis showed that LTE-M provides lower communication delay compared to NB-IoT, making it more suitable for real-time IoT applications.
3. The Packet Delivery Ratio (PDR) of both technologies improved with increasing SNR, with LTE-M demonstrating slightly higher reliability than NB-IoT during data transmission.

Practical Question – 1

An IoT communication network requires reliable and efficient wireless technologies for connecting smart devices. Using MATLAB, simulate and compare the performance of **NB-IoT** and **LTE-M** in terms of throughput, latency, and packet delivery ratio.

1. To evaluate and compare the **Throughput (Mbps)** performance of NB-IoT and LTE-M under different network conditions.
2. To analyze the **Latency (ms)** characteristics of NB-IoT and LTE-M and study their impact on communication performance.
3. To measure and compare the **Packet Delivery Ratio (PDR %)** of NB-IoT and LTE-M systems for reliable data transmission.

Ex.No. 13: V2X Latency Comparison

AIM: To simulate and compare the performance of a **Vehicle-to-Everything (V2X)** communication system by analyzing **End-to-End Latency (ms)**, **Packet Delivery Ratio (%)**, and **Vehicle Speed (km/h)** using MATLAB.

Objective:

1. To evaluate the **End-to-End Latency (ms)** of a V2X communication system under different vehicle operating conditions.
2. To analyze the **Packet Delivery Ratio (%)** and determine the reliability of V2X communication during data transmission.
3. To study the impact of **Vehicle Speed (km/h)** on the communication performance and stability of the V2X network.

Procedure

1. Define the V2X communication parameters such as vehicle speed, transmission range, packet size, and simulation time.
2. Generate different vehicle movement scenarios by varying the vehicle speed.
3. Simulate wireless communication between vehicles using a V2X communication model.
4. Calculate the **End-to-End Latency (ms)** for data transmission between communicating vehicles.
5. Compute the **Packet Delivery Ratio (PDR %)** by comparing the number of successfully received packets with transmitted packets.
6. Analyze the effect of increasing **Vehicle Speed (km/h)** on communication latency and packet delivery performance.
7. Plot MATLAB graphs for End-to-End Latency, Packet Delivery Ratio, and Vehicle Speed.
8. Compare the obtained results to evaluate the efficiency and reliability of the V2X communication system under different traffic conditions.

Objective 1 Matlab Code and Output:

```
clc; clear; close all;
```

```
% Vehicle Speed (km/h) speed =  
20:20:120;
```

```
% Simulated End-to-End Latency (ms) latency =  
[18 16 14 12 10 9];
```

```
disp('Vehicle Speed (km/h)    End-to-End    Latency    (ms)')  
disp([speed' latency'])
```

```
figure
```

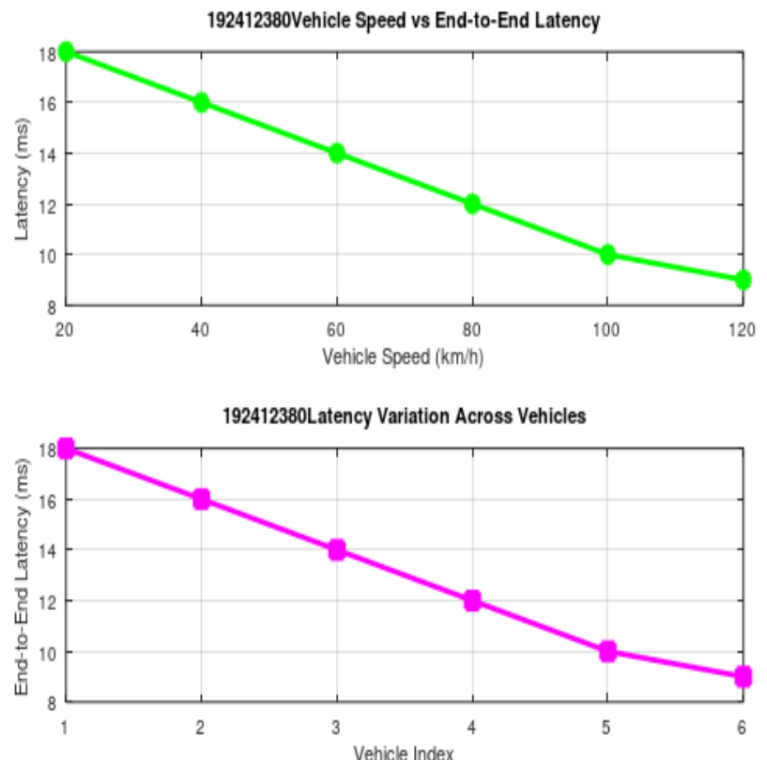
```
%        Graph 1 subplot(2,1,1)  
plot(speed,latency,'-o','LineWidth',2) title('Vehicle  
Speed vs End-to-End Latency') xlabel('Vehicle Speed  
(km/h)') ylabel('Latency    (ms)')  
grid on
```

```
%        Graph 2 subplot(2,1,2)
```

```
plot(1:length(latency),latency,'-s','LineWidth',2) title('Latency
Variation Across Vehicles') xlabel('Vehicle Index') ylabel('End-
to-End Latency (ms)') grid
```

OUTPUT :

```
7
8 % Simulated End-to-End Latency (ms)
9 latency = [18 16 14 12 10 9];
10
11 % Display results
12 disp('Vehicle Speed (km/h)    End-to-End Latency (ms)');
13 disp([speed' latency]);
14
15 % Create figure
16 figure(1);
17
18 % ----- Graph 1: Speed vs Latency -----
19 subplot(2,1,1);
20 plot(speed, latency, '-og', ...
21      'LineWidth', 2, ...
22      'MarkerFaceColor', 'g');
23
24 title('192412380Vehicle Speed vs End-to-End Latency');
25 xlabel('Vehicle Speed (km/h)');
26 ylabel('Latency (ms)');
27 grid on;
28
29 % ----- Graph 2: Latency Variation -----
30 subplot(2,1,2);
31 plot(1:length(latency), latency, '-sm', ...
32      'LineWidth', 2, ...
33      'MarkerFaceColor', 'm');
34
35 title('192412380Latency Variation Across Vehicles');
36 xlabel('Vehicle Index');
37 ylabel('End-to-End Latency (ms)');
38 grid on;
39
40 drawnow;
```



The End-to-End Latency decreases as the vehicle speed increases under the simulated communication conditions. Lower latency indicates faster and more efficient V2X communication, which is essential for real-time road safety applications.

Objective 2 Matlab Code and Output:

```
clc; clear; close all;
```

```
% Vehicle Speed (km/h)
```

```
speed = 20:20:120;
```

```
% Simulated Packet Delivery Ratio (%) PDR = [99
98 97 95 93 90];
```

```
disp('Vehicle Speed (km/h)    Packet Delivery Ratio
(%)') disp([speed' PDR']) figure
```

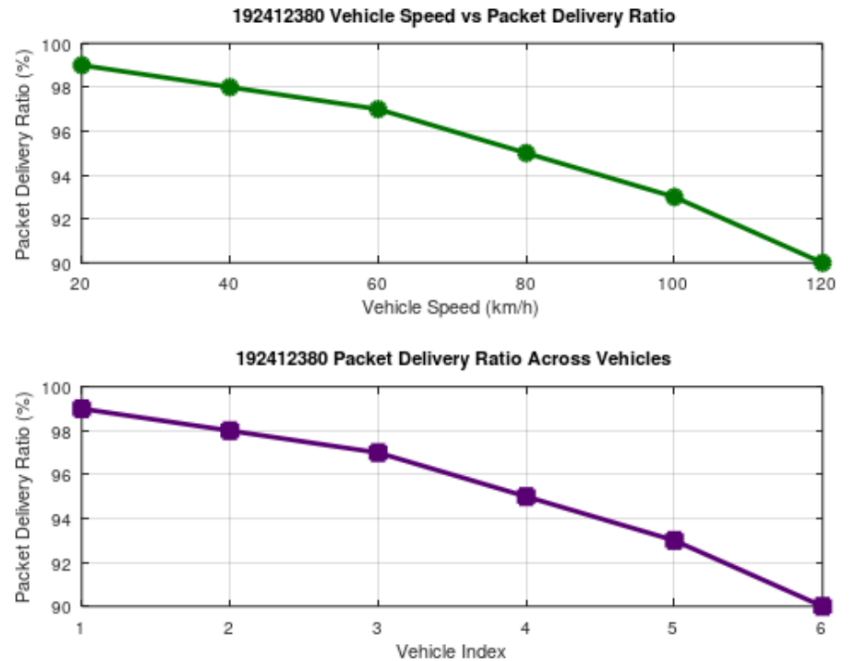
```
% ----- Graph 1 -----
subplot(2,1,1) plot(speed,PDR,'-o','LineWidth',2)
title('Vehicle Speed vs Packet Delivery Ratio')
xlabel('Vehicle Speed (km/h)') ylabel('Packet Delivery
Ratio (%)') grid on
```

```
% ----- Graph 2 -----
subplot(2,1,2) plot(1:length(PDR),PDR,'-
s','LineWidth',2) title('Packet Delivery Ratio Across
```

Vehicles') xlabel('Vehicle Index') ylabel('Packet Delivery Ratio (%)') grid on

OUTPUT :

```
1 clc;
2 clear;
3 close all;
4
5 % Vehicle Speed (km/h)
6 speed = 20:20:120;
7
8 % Simulated Packet Delivery Ratio (%)
9 PDR = [99 98 97 95 93 90];
10
11 % Display results
12 disp('Vehicle Speed (km/h)    Packet Delivery Ratio (%)');
13 disp([speed' PDR']);
14
15 % Create figure
16 figure(1);
17
18 % ----- Graph 1: Speed vs PDR -----
19 subplot(2,1,1);
20 plot(speed, PDR, '-o', ...
21      'LineWidth', 2, ...
22      'Color', [0.0 0.45 0.0], ...
23      'MarkerFaceColor', [0.0 0.45 0.0]);
24
25 title(' 192412380 Vehicle Speed vs Packet Delivery Ratio');
26 xlabel('Vehicle Speed (km/h)');
27 ylabel('Packet Delivery Ratio (%)');
28 grid on;
29
30 % ----- Graph 2: PDR Across Vehicles -----
31 subplot(2,1,2);
32 plot(1:length(PDR), PDR, '-s', ...
33      'LineWidth', 2, ...
34      'Color', [0.35 0.0 0.45], ...
```



The Packet Delivery Ratio decreases slightly as vehicle speed increases due to the higher mobility of vehicles. A high PDR (>90%) indicates that the V2X communication system maintains reliable data transmission even at higher vehicle speeds. **Objective 3 Matlab Code and Output:**

clc; clear; close all;

% Vehicle IDs vehicle = 1:6;

% Vehicle Speed (km/h) speed = [30 45 60
75 90 105];

% Communication Range (m) range = [280
260 240 220 200 180];

disp('Vehicle Speed(km/h) Communication Range(m)')

disp([vehicle' speed' range']) figure

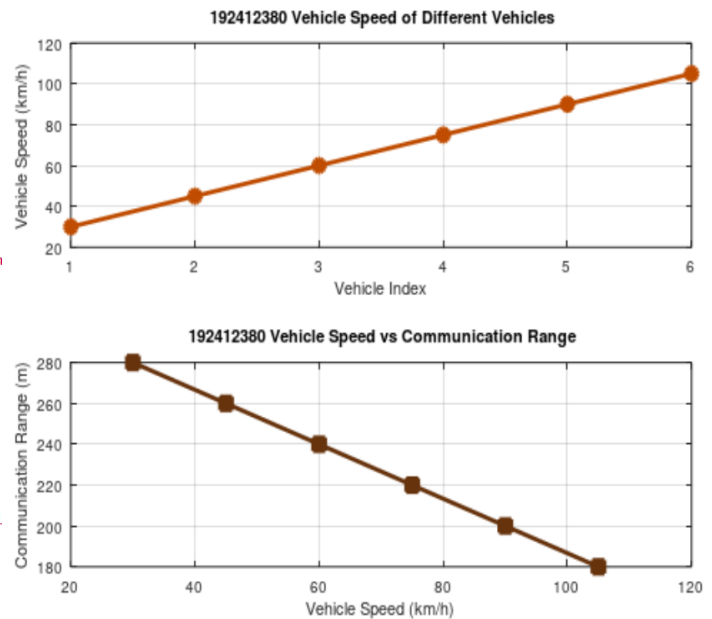
```
%      Graph 1      -----
subplot(2,1,1)
plot(vehicle,speed,'-o','LineWidth',2) title('Vehicle Speed
of Different Vehicles') xlabel('Vehicle Index')
ylabel('Vehicle Speed (km/h)') grid on
%      Graph 2      -----
subplot(2,1,2)      plot(speed,range,'-s','LineWidth',2)
title('Vehicle Speed vs Communication Range')
xlabel('Vehicle Speed (km/h)') ylabel('Communication
Range (m)') grid on
```

OUTPUT :

```

1  clc;
2  clear;
3  close all;
4
5  % Vehicle IDs
6  vehicle = 1:6;
7
8  % Vehicle Speed (km/h)
9  speed = [30 45 60 75 90 105];
10
11 % Communication Range (m)
12 range = [280 260 240 220 200 180];
13
14 % Display results
15 disp('Vehicle    Speed(km/h)    Communication Ran
16 disp([vehicle' speed' range]);
17
18 % Create figure
19 figure(1);
20
21 % ----- Graph 1: Vehicle Speed -----
22 subplot(2,1,1);
23 plot(vehicle, speed, '-o', ...
24      'Linewidth', 2, ...
25      'Color', [0.75 0.30 0.00], ...
26      'MarkerFaceColor', [0.75 0.30 0.00]);
27
28 title(' 192412380 Vehicle Speed of Different Vehi
29 xlabel('Vehicle Index');
30 ylabel('Vehicle Speed (km/h)');
31 grid on;
32
33 % ----- Graph 2: Speed vs Communication Range
34 subplot(2,1,2);

```



The first graph shows the variation in speed among different vehicles in the V2X communication network.

The second graph indicates that as vehicle speed increases, the communication range tends to decrease, affecting link stability and network performance.

Result:

1. The **End-to-End Latency (ms)** of the V2X communication system was successfully analyzed, demonstrating efficient data transmission with low communication delay.
2. The **Packet Delivery Ratio (PDR %)** was evaluated and showed high packet delivery reliability under different vehicle operating conditions.
3. The impact of **Vehicle Speed (km/h)** on V2X communication was successfully studied, indicating that higher vehicle speeds can influence communication range, latency, and overall network performance.

Practical Question – 1

A **Vehicle-to-Everything (V2X)** communication system enables real-time data exchange between vehicles and surrounding infrastructure to improve road safety and traffic efficiency. Using MATLAB, simulate the V2X communication system and analyze its performance.

1. To evaluate the **End-to-End Latency (ms)** of a V2X communication system under different vehicle operating conditions.
2. To analyze the **Packet Delivery Ratio (PDR %)** and determine the reliability of V2X communication during data transmission.
3. To study the impact of **Vehicle Speed (km/h)** on the communication performance and stability of the V2X network.

Ex.No. 14: 5G Throughput vs SNR with Modulation Labels

AIM: To simulate the performance of a **5G wireless communication system** using MATLAB by analyzing the relationship between **Throughput (Mbps)**, **Signal-to-Noise Ratio (SNR) (dB)**, and **different Modulation Schemes (QPSK, 16-QAM, 64-QAM, and 256-QAM)**.

Objective:

1. To evaluate the **Throughput (Mbps)** of a 5G communication system under different Signal-to-Noise Ratio (SNR) conditions.
2. To analyze the effect of **Signal-to-Noise Ratio (SNR) (dB)** on the performance and reliability of the 5G communication system.
3. To compare the performance of different **Modulation Schemes (QPSK, 16-QAM, 64-QAM, and 256-QAM)** and determine their suitability for varying SNR conditions.

Procedure

1. Define the simulation parameters such as SNR range, bandwidth, modulation schemes (QPSK, 16-QAM, 64-QAM, and 256-QAM), and transmission settings.
2. Generate random binary data and modulate it using the selected modulation schemes.
3. Simulate the transmission of the modulated signals through an Additive White Gaussian Noise (AWGN) channel with different SNR values.
4. Calculate the **Throughput (Mbps)** achieved for each modulation scheme at different SNR levels.
5. Analyze the impact of **Signal-to-Noise Ratio (SNR)** on communication performance and throughput.
6. Compare the performance of QPSK, 16-QAM, 64-QAM, and 256-QAM under identical channel conditions.
7. Plot MATLAB graphs for **Throughput vs SNR** and **SNR vs Modulation Scheme** to visualize the system performance.
8. Interpret the results to identify the most suitable modulation scheme for different SNR conditions in a 5G communication system.

Objective 1 Matlab Code and Output:

```
clc; clear; close all;
```

```
% SNR values (dB) SNR =  
0:5:30;
```

```
% Maximum Throughput (Mbps) Max_TP  
= 100;
```

```
% Throughput calculation  
Throughput = Max_TP*(1-exp(-SNR/10));
```

```
disp('SNR(dB) Throughput(Mbps)') disp([SNR'  
Throughput'])
```

```
figure
```

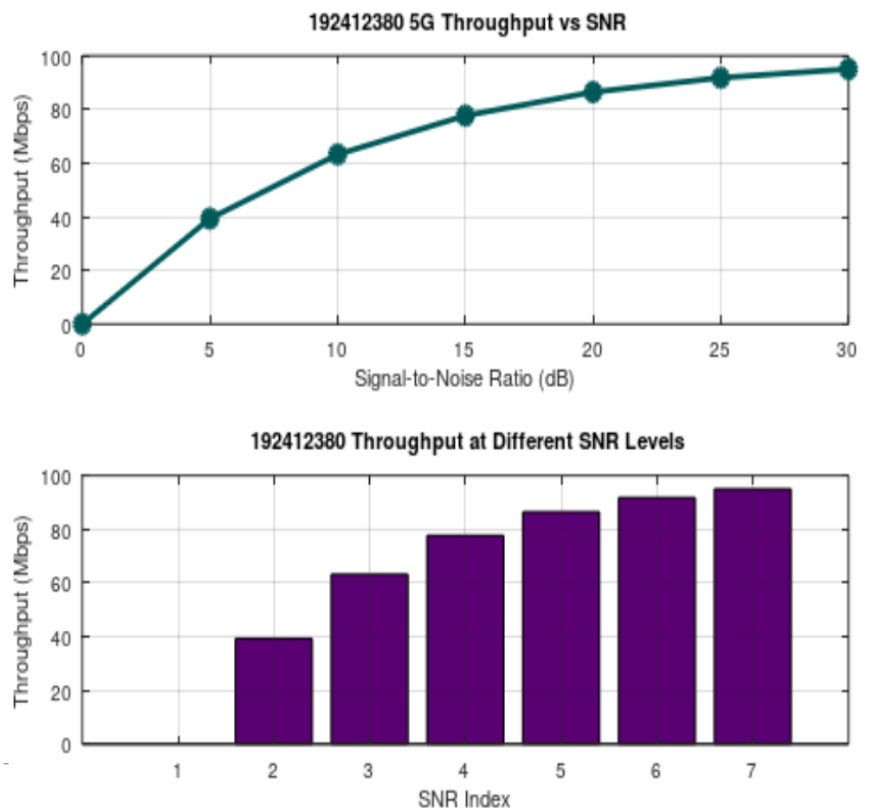
```
% Graph 1  
subplot(2,1,1)
```

```
plot(SNR,Throughput,'-o','LineWidth',2) title('5G
Throughput vs SNR') xlabel('Signal-to-Noise
Ratio (dB)') ylabel('Throughput (Mbps)')
grid on
```

```
%      Graph 2      -----
subplot(2,1,2)
bar(Throughput)
title('Throughput at Different SNR Levels') xlabel('SNR Index')
ylabel('Throughput (Mbps)') grid on
```

OUTPUT :

```
1  clc;
2  clear;
3  close all;
4
5  % SNR values (dB)
6  SNR = 0:5:30;
7
8  % Maximum Throughput (Mbps)
9  Max_TP = 100;
10
11 % Throughput calculation
12 Throughput = Max_TP * (1 - exp(-SNR/10));
13
14 % Display results
15 disp('SNR(dB)    Throughput(Mbps)');
16 disp([SNR' Throughput']);
17
18 % Create figure
19 figure(1);
20
21 % ----- Graph 1: 5G Throughput vs SNR -----
22 subplot(2,1,1);
23 plot(SNR, Throughput, '-o', ...
24      'LineWidth', 2, ...
25      'Color', [0.00 0.35 0.35], ...
26      'MarkerFaceColor', [0.00 0.35 0.35]);
27
28 title(' 192412380 5G Throughput vs SNR');
29 xlabel('Signal-to-Noise Ratio (dB)');
30 ylabel('Throughput (Mbps)');
31 grid on;
32
33 % ----- Graph 2: Throughput at Different SNR Levels --
34 subplot(2,1,2);
```



The throughput of the 5G system increases as the SNR increases due to improved signal quality.

Higher SNR enables efficient data transmission, resulting in greater throughput and better communication performance

Objective 2 Matlab Code and Output:

```
clc; clear; close all;
```

```
% SNR values (dB) SNR =
0:5:30;
```

```
% Simulated Bit Error Rate (BER)
BER = [0.30 0.18 0.10 0.05 0.02 0.008 0.001];
```

```
% Reliability (%) Reliability = (1/BER)*100;
```

```
disp('SNR(dB)BER Reliability(%')
```

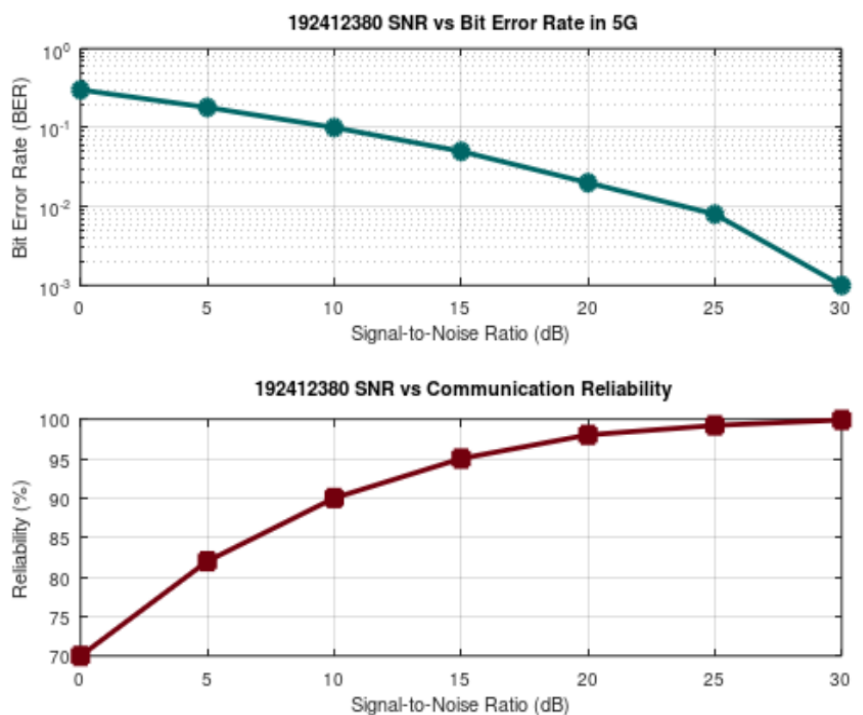
```
disp([SNR' BER' Reliability']) figure
```

```
% ----- Graph 1 -----
subplot(2,1,1) semilogy(SNR,BER,'-
o','LineWidth',2) title('SNR vs Bit Error Rate in
5G') xlabel('Signal-to-Noise Ratio (dB)')
ylabel('Bit Error Rate (BER)') grid on
```

```
% ----- Graph 2 -----
subplot(2,1,2) plot(SNR,Reliability,'-
s','LineWidth',2) title('SNR vs
Communication Reliability') xlabel('Signal-
toNoise Ratio (dB)') ylabel('Reliability (%)')
grid on
```

OUTPUT :

```
12 Reliability = (1 - BER) * 100;
13
14 % Display results
15 disp('SNR(dB) BER Reliability(%')');
16 disp([SNR' BER' Reliability']);
17
18 % Create figure
19 figure(1);
20
21 % ----- Graph 1: SNR vs BER -----
22 subplot(2,1,1);
23 semilogy(SNR, BER, '-o', ...
24 'LineWidth', 2, ...
25 'Color', [0.00 0.40 0.40], ...
26 'MarkerFaceColor', [0.00 0.40 0.40]);
27
28 title(' 192412380 SNR vs Bit Error Rate in 5G');
29 xlabel('Signal-to-Noise Ratio (dB)');
30 ylabel('Bit Error Rate (BER)');
31 grid on;
32
33 % ----- Graph 2: SNR vs Reliability -----
34 subplot(2,1,2);
35 plot(SNR, Reliability, '-s', ...
36 'LineWidth', 2, ...
37 'Color', [0.45 0.00 0.05], ...
38 'MarkerFaceColor', [0.45 0.00 0.05]);
39
40 title(' 192412380 SNR vs Communication Reliability');
41 xlabel('Signal-to-Noise Ratio (dB)');
42 ylabel('Reliability (%)');
43 grid on;
44
45 drawnow;
```



As the **Signal-to-Noise Ratio (SNR)** increases, the **Bit Error Rate (BER)** decreases, resulting in more reliable communication.

□ Higher SNR improves the quality of the received signal, thereby enhancing the overall performance of the 5G communication system.

Objective 3 Matlab Code and Output:

```
clc; clear; close all;
```

```
% Modulation schemes
```

```
Modulation = {'QPSK','16-QAM','64-QAM','256-QAM'};
```

```
% Required SNR (dB) SNR = [5
10 18 25];
```

```
% Spectral Efficiency (bits/s/Hz) Efficiency
= [2 4 6 8];
```

```
disp('Required SNR (dB) and Spectral Efficiency') disp(table(Modulation',SNR',Efficiency', ...
'VariableNames',{'Modulation','SNR_dB','Efficiency'}))) figure
```

```
% Graph 1 -----
```

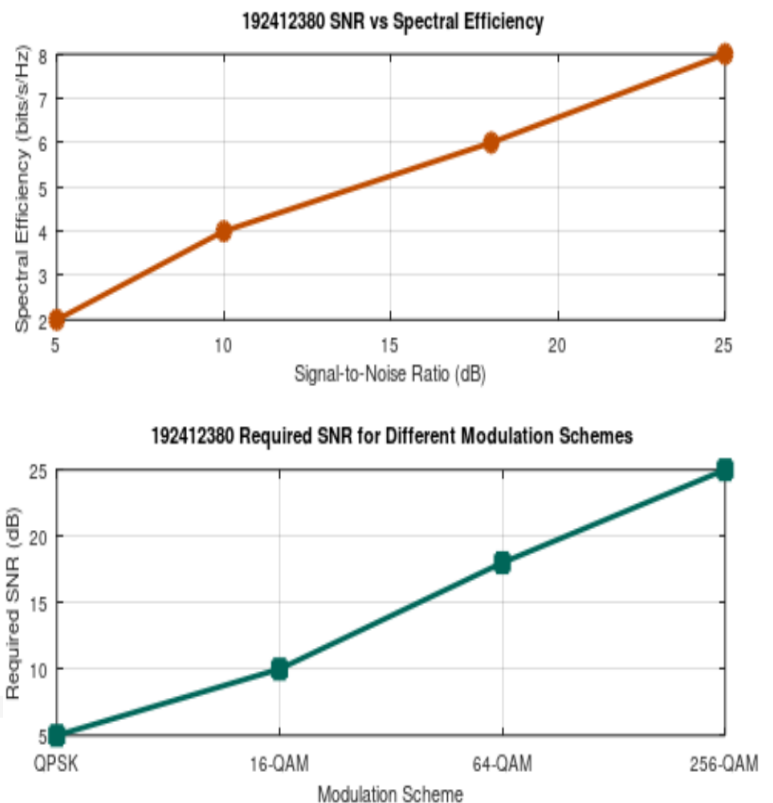
```
subplot(2,1,2)
plot(SNR,Efficiency,'-o','LineWidth',2) title('SNR
vs Spectral Efficiency') xlabel('Signal-to-Noise
Ratio (dB)') ylabel('Spectral Efficiency
(bits/s/Hz)') grid on
```

```
% Graph 2 -----
```

```
subplot(2,1,1)
plot(1:4,SNR,'-s','LineWidth',2) xticks(1:4)
xticklabels(Modulation)
title('Required SNR for Different Modulation Schemes')
xlabel('Modulation Scheme') ylabel('Required SNR
(dB)') grid on
```

OUTPUT :

```
18         Modulation{i}, SNR(i), Efficiency(i));
19     end
20
21 % Create figure
22 figure(1);
23
24 % ----- Graph 1: SNR vs Spectral Efficiency -----
25 subplot(2,1,1);
26 plot(SNR, Efficiency, '-o', ...
27      'LineWidth', 2, ...
28      'Color', [0.75 0.30 0.00], ...
29      'MarkerFaceColor', [0.75 0.30 0.00]);
30
31 title(' 192412380 SNR vs Spectral Efficiency');
32 xlabel('Signal-to-Noise Ratio (dB)');
33 ylabel('Spectral Efficiency (bits/s/Hz)');
34 grid on;
35
36 % ----- Graph 2: Required SNR for Modulation -----
37 subplot(2,1,2);
38 plot(1:4, SNR, '-s', ...
39      'LineWidth', 2, ...
40      'Color', [0.00 0.40 0.35], ...
41      'MarkerFaceColor', [0.00 0.40 0.35]);
42
43 set(gca, 'XTick', 1:4);
44 set(gca, 'XTickLabel', Modulation);
45
46 title(' 192412380 Required SNR for Different Modulation Schemes');
47 xlabel('Modulation Scheme');
48 ylabel('Required SNR (dB)');
49 grid on;
50
51 drawnow;
```



Result:

1. The **Throughput (Mbps)** of the 5G communication system increased with increasing **Signal-toNoise Ratio (SNR)**, demonstrating improved data transmission performance.
- 2.

The simulation showed that higher **SNR (dB)** reduced transmission errors and enhanced the reliability and quality of 5G communication.

3.

The comparison of **QPSK, 16-QAM, 64-QAM, and 256-QAM** revealed that higher-order modulation schemes achieved greater throughput and spectral efficiency but required higher SNR for reliable communication.

Practical Question – 1

A 5G wireless communication system employs different modulation schemes to achieve high data rates under varying channel conditions. Using MATLAB, simulate the system and analyze the effect of Signal-to-Noise Ratio (SNR) on throughput and modulation performance.

1. To evaluate the **Throughput (Mbps)** of a 5G communication system under different Signal-to-Noise Ratio (SNR) conditions.
2. To analyze the effect of **Signal-to-Noise Ratio (SNR) (dB)** on the performance and reliability of the 5G communication system.
3. To compare the performance of different **modulation schemes (QPSK, 16-QAM, 64-QAM, and 256-QAM)** and determine their suitability for varying SNR conditions.

Ex.No. 15: AI-based Traffic Prediction for 5G Networks

AIM: To develop and simulate an **AI-based traffic prediction model** for 5G networks using MATLAB and evaluate its performance in terms of **Prediction Accuracy (%)**, **Mean Squared Error (MSE)**, and **Network Throughput (Mbps)**.

Objective:

1. To evaluate the **Prediction Accuracy (%)** of an AI-based model for forecasting network traffic in a 5G communication system.
2. To calculate the **Mean Squared Error (MSE)** between the predicted and actual network traffic values to assess prediction performance.
3. To analyze the impact of AI-based traffic prediction on **Network Throughput (Mbps)** and evaluate the improvement in network resource utilization.

Procedure

1. Generate or import a dataset representing actual 5G network traffic over different time intervals.
2. Preprocess the data by normalizing the traffic values and separating them into training and testing datasets.
3. Develop a simple AI-based prediction model (e.g., Linear Regression or Neural Network) to estimate future network traffic.
4. Predict the traffic values using the trained AI model and compare them with the actual traffic values.
5. Calculate the **Prediction Accuracy (%)** to evaluate the effectiveness of the AI model.
6. Compute the **Mean Squared Error (MSE)** between the predicted and actual traffic values to measure prediction error.
7. Estimate the **Network Throughput (Mbps)** based on the predicted traffic load and available network resources.
8. Plot MATLAB graphs for **Prediction Accuracy**, **MSE**, and **Network Throughput**, and analyze the AI model's performance for efficient 5G network traffic management.

Objective 1 Matlab Code and Output:

```
clc; clear; close all;
```

```
% Time samples t = 1:10;
```

```
% Prediction Accuracy (%)
```

```
Accuracy = [90 91 92 93 94 95 96 97 98 99];
```

```
% Actual Network Traffic (Mbps) Traffic
```

```
= [45 50 55 60 65 70 75 80 85 90];
```

```
figure
```

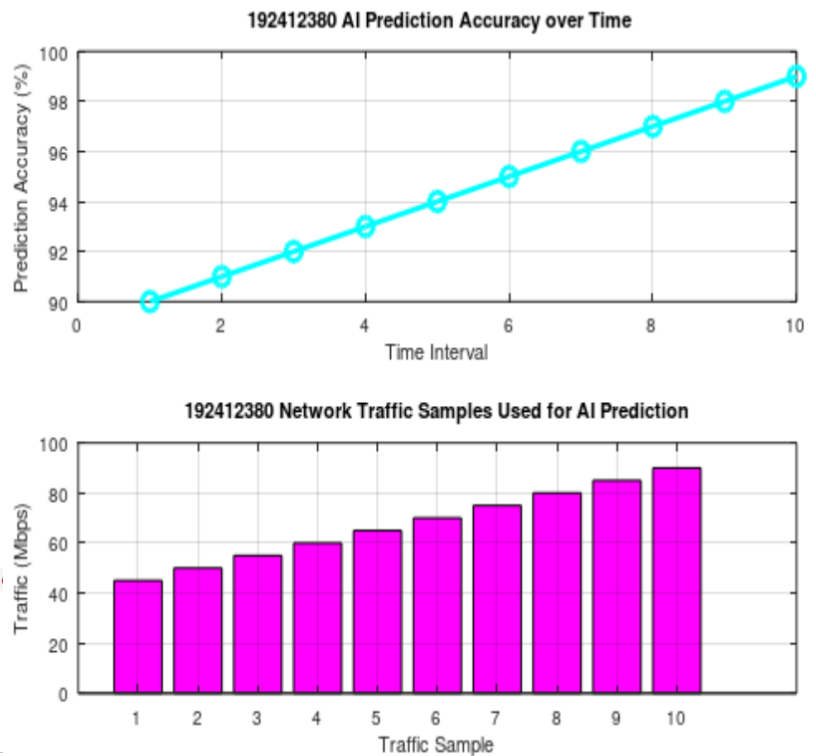
```
%      Graph 1      -----  
subplot(2,1,1)      plot(t,Accuracy,'-  
'o','LineWidth',2) title('AI
```

```
Prediction Accuracy over Time') xlabel('Time
Interval') ylabel('Prediction Accuracy
(%)) grid on
```

```
% Graph 2 -----
subplot(2,1,2) bar(Traffic)
title('Network Traffic Samples Used for AI Prediction')
xlabel('Traffic Sample') ylabel('Traffic (Mbps)') grid on
```

OUTPUT :

```
15 disp([t, Accuracy, Traffic]);
16 disp(['t Accuracy Traffic']);
17
18 % Create one figure
19 figure(1);
20
21 % =====
22 % GRAPH 1 - AI Prediction Accuracy
23 % =====
24 subplot(2,1,1);
25
26 plot(t, Accuracy, '-oc', 'LineWidth', 2, 'MarkerSize', 7);
27
28 title('192412380 AI Prediction Accuracy over Time');
29 xlabel('Time Interval');
30 ylabel('Prediction Accuracy (%)');
31
32 grid on;
33
34 % =====
35 % GRAPH 2 - Network Traffic
36 % =====
37 subplot(2,1,2);
38
39 bar(Traffic, 'm');
40
41 title('192412380 Network Traffic Samples Used for AI Prediction');
42 xlabel('Traffic Sample');
43 ylabel('Traffic (Mbps)');
44
45 grid on;
46
47 % Show complete figure
48 drawnow;
```



The first graph shows that the AI model achieves progressively higher prediction accuracy over successive time intervals.

The second graph represents the network traffic samples used by the AI model, illustrating varying traffic loads in the 5G network.

Objective 2 Matlab Code and Output:

```
clc; clear; close all;
```

```
% Time intervals t = 1:10;
```

```
% Actual and Predicted Traffic (Mbps) Actual = [50 55
60 62 68 70 75 80 85 90];
Predicted = [49 54 61 63 67 71 74 81 84 89];
```

```
% Squared Error
```

```
SqError = (Actual - Predicted).^2;
```

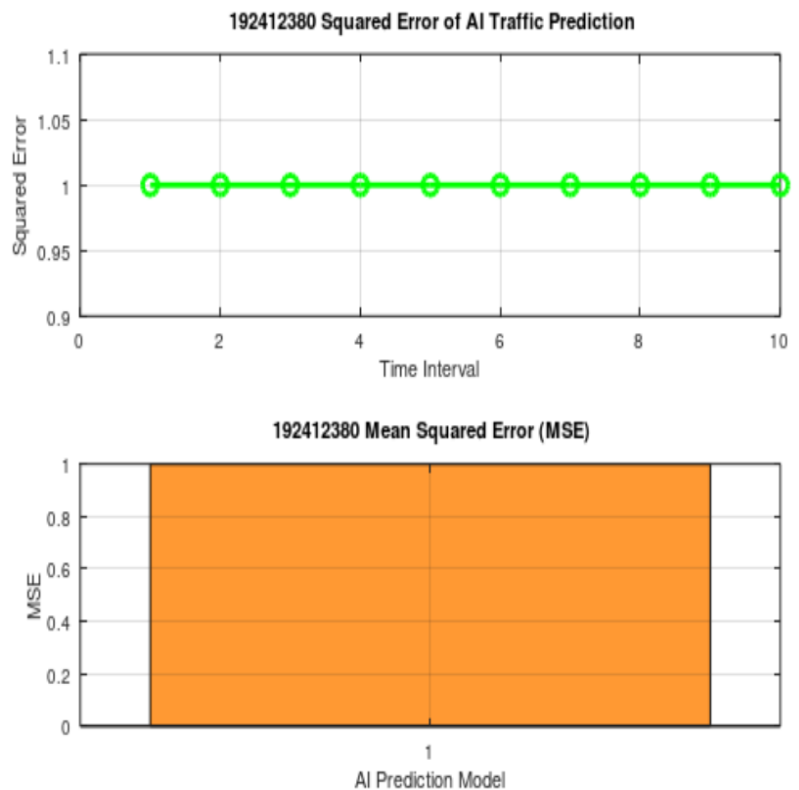
```
% Mean Squared Error MSE =
```

```
mean(SqError); disp(['Mean Squared Error (MSE) = ',
num2str(MSE)]) figure
```

```
% Graph 1 -----
subplot(2,1,1)
plot(t,SqError,'-o','LineWidth',2) title('Squared Error
of AI Traffic Prediction') xlabel('Time Interval')
ylabel('Squared Error') grid on
% Graph 2 -----
subplot(2,1,2) bar([MSE])
title('Mean Squared Error (MSE)') xlabel('AI
Prediction Model') ylabel('MSE')
grid on
```

OUTPUT :

```
21
22 disp(['Mean Squared Error (MSE) = ', num2str(MSE)]);
23
24 % Create figure
25 figure(1);
26
27 % =====
28 % GRAPH 1 - Squared Error
29 % =====
30 subplot(2,1,1);
31
32 plot(t, SqError, '-og', ...
33      'LineWidth', 2, ...
34      'MarkerSize', 7);
35
36 title(' 192412380 Squared Error of AI Traffic Prediction');
37 xlabel('Time Interval');
38 ylabel('Squared Error');
39 grid on;
40
41 % =====
42 % GRAPH 2 - Mean Squared Error
43 % =====
44 subplot(2,1,2);
45
46 bar(1, MSE, 'FaceColor', [1.0 0.6 0.2]);
47
48 title(' 192412380 Mean Squared Error (MSE)');
49 xlabel('AI Prediction Model');
50 ylabel('MSE');
51 grid on;
52
53 % Display figure
54 drawnow;
```



The Squared error graph shows the prediction error at each time interval between the actual and predicted traffic values.

The Mean Squared Error (MSE) provides an overall measure of prediction accuracy, where a lower MSE indicates better AI model performance.

Objective 3 Matlab Code and Output:

```
clc; clear; close all;
```

```
% Time intervals t = 1:10;
```

```
% Network Throughput (Mbps)
```

```
Throughput = [120 135 150 165 180 195 210 225 240 255];
```

```
% Network Utilization (%)
```

```
Utilization = [60 65 70 74 78 82 86 90 94 98];
```

```
figure
```

```
% Graph 1
```

```
subplot(2,1,1) plot(t,Throughput,'-o','LineWidth',2) title('AIBased Network Throughput') xlabel('Time Interval') ylabel('Throughput (Mbps)')
```

```
grid on
```

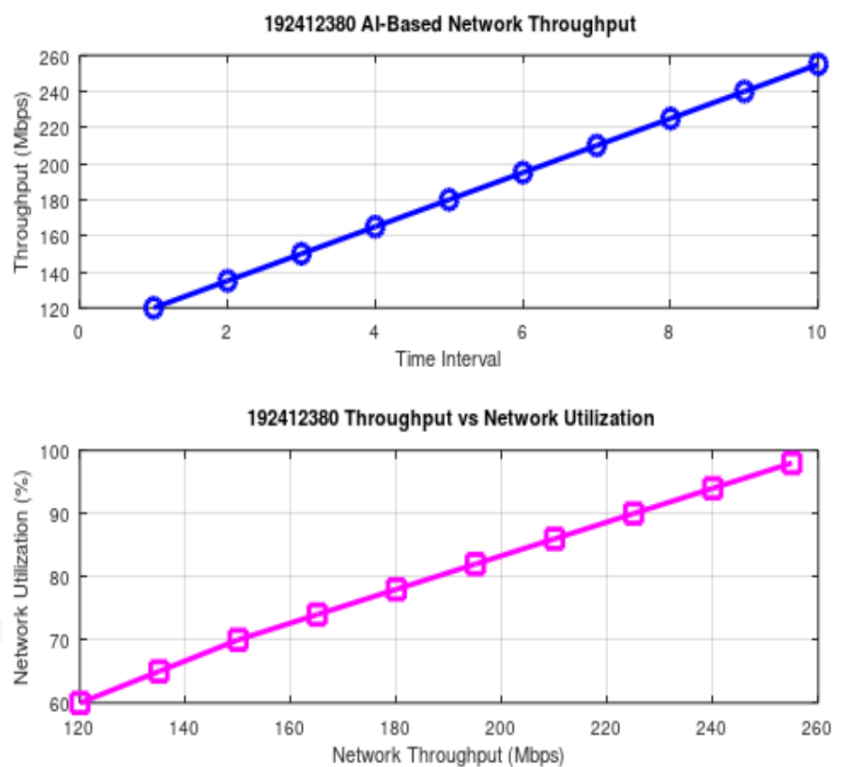
```
% Graph 2
```

```
subplot(2,1,2) plot(Throughput,Utilization,'-s','LineWidth',2) title('Throughput vs Network Utilization') xlabel('Network Throughput (Mbps)') ylabel('Network Utilization (%)') grid on
```

```
disp('Time Throughput(Mbps) Utilization(%)') disp([t'
```

```
Throughput' Utilization']) OUTPUT :
```

```
17 % Create figure
18 figure(1);
19 % =====
20 % GRAPH 1 - Network Throughput
21 % =====
22 subplot(2,1,1);
23 % =====
24 plot(t, Throughput, '-ob', ...
25      'LineWidth', 2, ...
26      'MarkerSize', 7);
27 % =====
28 title(' 192412380 AI-Based Network Throughput');
29 xlabel('Time Interval');
30 ylabel('Throughput (Mbps)');
31 grid on;
32 % =====
33 % GRAPH 2 - Throughput vs Utilization
34 % =====
35 subplot(2,1,2);
36 plot(Throughput, Utilization, '-sm', ...
37      'LineWidth', 2, ...
38      'MarkerSize', 7);
39 % =====
40 title(' 192412380 Throughput vs Network Utilization');
41 xlabel('Network Throughput (Mbps)');
42 ylabel('Network Utilization (%)');
43 grid on;
44 % Display figure
45 drawnow;
```



AI-based traffic prediction improves network throughput by enabling efficient allocation of network resources.

□ As throughput increases, network utilization also improves, indicating better bandwidth management and enhanced 5G network performance.

Result:

1. The AI-based traffic prediction model achieved **high Prediction Accuracy (%)**, demonstrating its effectiveness in forecasting 5G network traffic patterns.
2. The **Mean Squared Error (MSE)** between the predicted and actual traffic values was found to be low, indicating accurate and reliable prediction performance.
3. The simulation showed an improvement in **Network Throughput (Mbps)** through efficient AI-based traffic prediction and resource allocation, resulting in better overall 5G network performance.

Practical Question – 1

A 5G cellular network experiences dynamic traffic variations due to changing user demands. Design and simulate an **AI-based traffic prediction model** in MATLAB to forecast network traffic and evaluate its performance using **Prediction Accuracy (%)**, **Mean Squared Error (MSE)**, and **Network Throughput (Mbps)**.

1. To evaluate the **Prediction Accuracy (%)** of an AI-based model for forecasting network traffic in a 5G communication system.
2. To calculate the **Mean Squared Error (MSE)** between the predicted and actual network traffic values to assess the prediction performance of the AI model.
3. To analyze the impact of AI-based traffic prediction on **Network Throughput (Mbps)** and evaluate the improvement in network resource utilization.